

材料边界：本地、离线工程原型；Web 仅展示已生成结果；单图推理需模型环境；不提供在线推理或生产调度服务。

```
# --- run.py ---
1  """Project-level runner for the robot tunnel inspection application."""
2
3  from __future__ import annotations
4
5  import argparse
6  from pathlib import Path
7  from typing import Any
8
9
10 ROOT = Path(__file__).resolve().parent
11
12
13 def parse_args() -> argparse.Namespace:
14     parser = argparse.ArgumentParser(description="Run project-level workflows.")
15     parser.add_argument(
16         "--mode",
17         choices=["full_pipeline"],
18         default="full_pipeline",
19         help="Workflow mode. full_pipeline runs the multi-stage Orchestrator DAG.",
20     )
21     return parser.parse_args()
22
23
24 def run_full_pipeline(project_root: Path = ROOT) -> dict[str, Any]:
25     """Run the full pipeline through the multi-stage Orchestrator DAG."""
26
27     from orchestrator.dag.builder import build_dag
28     from orchestrator.executor import DAGExecutor
29     from orchestrator.registry import build_default_registry
30
31     dag_path = project_root / "config" / "dag.yaml"
32     if not dag_path.exists():
33         dag_path = ROOT / "config" / "dag.yaml"
34     tasks, config = build_dag(dag_path)
35     shared = dict(config.get("shared", {}))
36     shared.setdefault("project_root", str(project_root))
37     shared.setdefault("dag_config", str(dag_path))
38     context = {
39         "inputs": config.get("inputs", {}),
40         "outputs": {},
41         "shared": shared,
42         "task_status": {},
43     }
44
45     executor = DAGExecutor(build_default_registry(), project_root, dag_config=str(dag_path))
46     result = executor.run(tasks, context)
47     outputs = result.get("outputs", {})
48     return {
49         "engineering_rows": outputs.get("engineering_report", {}).get("engineering_rows", 0),
```

```

50     "growth_rows": outputs.get("growth_analysis", {}).get("growth_rows", 0),
51     "memory_rows": outputs.get("memory", {}).get("memory_bank_rows", 0),
52     "association_rows": outputs.get("association", {}).get("association_rows", 0),
53     "association_matched_rows": outputs.get("association", {}).get("association_matched_rows", 0),
54     "recheck_rows": outputs.get("visualization", {}).get("recheck_rows", 0),
55     "chart_count": outputs.get("visualization", {}).get("chart_count", 0),
56     "outputs": {
57         "disease_memory_bank": outputs.get("memory", {}).get("memory_bank_path"),
58         "disease_association_records": outputs.get("association", {}).get("association_records_path"),
59         "disease_growth_results": outputs.get("growth_analysis", {}).get("growth_results_path"),
60         "final_project_report": outputs.get("final_report", {}).get("final_project_report_path"),
61         "system_summary": outputs.get("final_report", {}).get("system_summary_path"),
62         "key_insights": outputs.get("final_report", {}).get("key_insights_path"),
63     },
64     "run_id": executor.run_id,
65     "run_dir": str(executor.run_info.run_dir),
66     "task_status": result.get("task_status", {}),
67 }
68
69
70 def run_full_pipeline_direct(project_root: Path = ROOT) -> dict[str, Any]:
71     """Compatibility alias for older callers; use run_full_pipeline instead."""
72
73     return run_full_pipeline(project_root)
74
75
76 def main() -> int:
77     args = parse_args()
78     if args.mode == "full_pipeline":
79         result = run_full_pipeline(ROOT)
80         print("机器人隧道巡检病害监测完整 pipeline 运行完成")
81         print(f"engineering rows: {result['engineering_rows']}")
82         print(f"memory rows: {result['memory_rows']}")
83         print(f"association rows: {result['association_rows']}")
84         print(f"growth rows: {result['growth_rows']}")
85         print(f"recheck rows: {result['recheck_rows']}")
86         print(f"visual artifacts: {result['chart_count']}")
87         print(f"final report: {result['outputs']['final_project_report']}")
88         return 0
89     raise ValueError(f"Unsupported mode: {args.mode}")
90
91
92 if __name__ == "__main__":
93     raise SystemExit(main())
# --- scripts/extract_kict_mask_features.py ---
1 from __future__ import annotations
2
3 import argparse
4 import csv
5 from pathlib import Path

```

```
6
7 import numpy as np
8 from PIL import Image
9
10
11 IMAGE_SUFFIXES = {".jpg", ".jpeg", ".png", ".bmp"}
12 MASK_SUFFIXES = {".png", ".jpg", ".jpeg", ".bmp"}
13 OUTPUT_COLUMNS = [
14     "image_file",
15     "mask_file",
16     "area_px",
17     "bbox_x1",
18     "bbox_y1",
19     "bbox_x2",
20     "bbox_y2",
21     "center_x",
22     "center_y",
23     "mask_width",
24     "mask_height",
25 ]
26
27
28 def parse_args() -> argparse.Namespace:
29     parser = argparse.ArgumentParser(description="Extract geometry features from KICT crack masks.")
30     parser.add_argument("--dataset-root", type=Path, required=True, help="KICT dataset root containing images/ and masks/.")
31     parser.add_argument(
32         "--output-csv",
33         type=Path,
34         default=Path("data/simulated/kict_mask_features.csv"),
35         help="CSV path for extracted mask geometry features.",
36     )
37     return parser.parse_args()
38
39
40 def find_dataset_dirs(dataset_root: Path) -> tuple[Path, Path]:
41     image_dir = dataset_root / "images"
42     mask_dir = dataset_root / "masks"
43     if not image_dir.is_dir():
44         raise FileNotFoundError(f"images folder not found: {image_dir}")
45     if not mask_dir.is_dir():
46         raise FileNotFoundError(f"masks folder not found: {mask_dir}")
47     return image_dir, mask_dir
48
49
50 def collect_files(folder: Path, suffixes: set[str]) -> dict[str, Path]:
51     files: dict[str, Path] = {}
52     for path in sorted(folder.iterdir()):
53         if path.is_file() and path.suffix.lower() in suffixes:
54             files.setdefault(path.stem, path)
55     return files
```

```
56
57
58 def relative_text(path: Path, root: Path) -> str:
59     try:
60         return path.relative_to(root).as_posix()
61     except ValueError:
62         return path.as_posix()
63
64
65 def extract_mask_features(mask_path: Path) -> dict[str, int | float]:
66     mask = np.array(Image.open(mask_path).convert("L")) > 0
67     area = int(mask.sum())
68     if area == 0:
69         return {
70             "area_px": 0,
71             "bbox_x1": -1,
72             "bbox_y1": -1,
73             "bbox_x2": -1,
74             "bbox_y2": -1,
75             "center_x": -1,
76             "center_y": -1,
77             "mask_width": 0,
78             "mask_height": 0,
79         }
80
81     ys, xs = np.nonzero(mask)
82     x1 = int(xs.min())
83     y1 = int(ys.min())
84     x2 = int(xs.max()) + 1
85     y2 = int(ys.max()) + 1
86     return {
87         "area_px": area,
88         "bbox_x1": x1,
89         "bbox_y1": y1,
90         "bbox_x2": x2,
91         "bbox_y2": y2,
92         "center_x": round((x1 + x2 - 1) / 2, 2),
93         "center_y": round((y1 + y2 - 1) / 2, 2),
94         "mask_width": x2 - x1,
95         "mask_height": y2 - y1,
96     }
97
98
99 def build_rows(dataset_root: Path) -> list[dict]:
100     image_dir, mask_dir = find_dataset_dirs(dataset_root)
101     image_files = collect_files(image_dir, IMAGE_SUFFIXES)
102     mask_files = collect_files(mask_dir, MASK_SUFFIXES)
103     rows = []
104
105     for stem, mask_path in sorted(mask_files.items()):
```

```
106         image_path = image_files.get(stem)
107         features = extract_mask_features(mask_path)
108         rows.append(
109             {
110                 "image_file": relative_text(image_path, dataset_root) if image_path else "",
111                 "mask_file": relative_text(mask_path, dataset_root),
112                 **features,
113             }
114         )
115     return rows
116
117
118 def write_csv(rows: list[dict], output_csv: Path) -> None:
119     output_csv.parent.mkdir(parents=True, exist_ok=True)
120     with output_csv.open("w", newline="", encoding="utf-8-sig") as file:
121         writer = csv.DictWriter(file, fieldnames=OUTPUT_COLUMNS)
122         writer.writeheader()
123         writer.writerows(rows)
124
125
126 def main() -> None:
127     args = parse_args()
128     rows = build_rows(args.dataset_root)
129     write_csv(rows, args.output_csv)
130     empty_count = sum(1 for row in rows if row["area_px"] == 0)
131     print(f"mask_count: {len(rows)}")
132     print(f"empty_mask_count: {empty_count}")
133     print(f"output_csv: {args.output_csv}")
134
135
136 if __name__ == "__main__":
137     main()
# --- scripts/merge_kict_with_simulation.py ---
1  from __future__ import annotations
2
3  import argparse
4  import csv
5  import sys
6  from collections import Counter
7  from pathlib import Path
8
9
10  DEFAULT_SIM_DIR = Path("data/simulated")
11  FALLBACK_SIM_DIR = Path("tunnel_defect_simulation/data/simulated")
12  DEFAULT_OUTPUT_CSV = Path("data/simulated/robot_kict_frame_records.csv")
13  DEFAULT_REPORT = Path("outputs/robot_kict_merge_report.md")
14
15  OUTPUT_COLUMNS = [
16      "image_id",
17      "inspection_id",
```

```
18     "frame_id",
19     "timestamp",
20     "mileage_m",
21     "mileage_text",
22     "ring_id",
23     "clock_direction",
24     "disease_id",
25     "disease_type",
26     "sim_area_px",
27     "sim_length_m",
28     "sim_width_mm",
29     "kict_image_file",
30     "kict_mask_file",
31     "kict_image_path",
32     "kict_mask_path",
33     "kict_area_px",
34     "kict_bbox_x1",
35     "kict_bbox_y1",
36     "kict_bbox_x2",
37     "kict_bbox_y2",
38     "kict_center_x",
39     "kict_center_y",
40     "kict_mask_width",
41     "kict_mask_height",
42     "has_crack",
43     "observation_source",
44     "comparability_status",
45 ]
46
47
48 def parse_args() -> argparse.Namespace:
49     parser = argparse.ArgumentParser(description="Merge robot simulation frames with KICT mask geometry features.")
50     parser.add_argument("--sim-dir", type=Path, default=DEFAULT_SIM_DIR, help="Directory containing simulation CSV files.")
51     parser.add_argument(
52         "--kict-features",
53         type=Path,
54         default=DEFAULT_SIM_DIR / "kict_mask_features.csv",
55         help="KICT mask feature CSV.",
56     )
57     parser.add_argument("--output-csv", type=Path, default=DEFAULT_OUTPUT_CSV, help="Merged robot/KICT output CSV.")
58     parser.add_argument("--report-file", type=Path, default=DEFAULT_REPORT, help="Markdown merge report output.")
59     args = parser.parse_args()
60     args.allow_default_fallback = not any(
61         arg == "--sim-dir" or arg.startswith("--sim-dir=") for arg in sys.argv[1:]
62     )
63     return args
64
65
66 def resolve_input_file(sim_dir: Path, filename: str, allow_fallback: bool = False) -> Path:
67     preferred = sim_dir / filename
```

```
68     if preferred.exists():
69         return preferred
70
71     # 默认目录缺文件时, 兼容当前仓库把仿真表放在子目录的历史布局。
72     if allow_fallback:
73         fallback = FALLBACK_SIM_DIR / filename
74         if fallback.exists():
75             return fallback
76
77     raise FileNotFoundError(f"Input file not found: {preferred}")
78
79
80 def read_csv(path: Path) -> list[dict[str, str]]:
81     if not path.exists():
82         raise FileNotFoundError(f"Input file not found: {path}")
83     with path.open(encoding="utf-8-sig", newline="") as file:
84         rows = list(csv.DictReader(file))
85     if not rows:
86         raise ValueError(f"Input CSV is empty: {path}")
87     return rows
88
89
90 def truthy(value: str | None) -> bool:
91     return str(value or "").strip().lower() in {"true", "1"}
92
93
94 def int_value(row: dict[str, str], key: str) -> int:
95     return int(float(row.get(key, "0") or 0))
96
97
98 def effective_has_crack(row: dict[str, str]) -> bool:
99     if "has_crack" in row and str(row.get("has_crack", "")).strip() != "":
100         return truthy(row.get("has_crack"))
101     return int_value(row, "area_px") > 0
102
103
104 def normalize_kict_path(row: dict[str, str], file_key: str, path_key: str) -> str:
105     # Older feature tables only have image_file/mask_file; treat them as usable relative paths.
106     return row.get(path_key) or row.get(file_key) or ""
107
108
109 def filter_valid_kict_samples(rows: list[dict[str, str]]) -> list[dict[str, str]]:
110     valid = [row for row in rows if effective_has_crack(row) and int_value(row, "area_px") > 0]
111     if not valid:
112         raise ValueError("kict_mask_features.csv has no valid crack samples")
113     return valid
114
115
116 def build_records(
117     inspection_rows: list[dict[str, str]],
```

```
118     mapping_rows: list[dict[str, str]],
119     valid_kict_rows: list[dict[str, str]],
120 ) -> tuple[list[dict[str, str]], int, list[str]]:
121     inspection_by_image = {row["image_id"]: row for row in inspection_rows}
122     output_rows: list[dict[str, str]] = []
123     missing_inspection_count = 0
124     missing_image_ids: list[str] = []
125
126     for index, mapping in enumerate(mapping_rows):
127         kict = valid_kict_rows[index % len(valid_kict_rows)]
128         inspection = inspection_by_image.get(mapping.get("image_id", ""))
129         if inspection is None:
130             missing_inspection_count += 1
131             missing_image_ids.append(mapping.get("image_id", ""))
132             inspection = {}
133
134     output_rows.append(
135         {
136             "image_id": mapping.get("image_id", ""),
137             "inspection_id": mapping.get("inspection_id", ""),
138             "frame_id": mapping.get("frame_id", ""),
139             # 工程元数据优先来自 inspection_sequence。
140             "timestamp": inspection.get("timestamp", ""),
141             "mileage_m": inspection.get("mileage_m", ""),
142             "mileage_text": inspection.get("mileage_text", ""),
143             "ring_id": inspection.get("ring_id", ""),
144             "clock_direction": inspection.get("clock_direction", ""),
145             "disease_id": mapping.get("disease_id", ""),
146             "disease_type": mapping.get("disease_type", ""),
147             "sim_area_px": mapping.get("area_px", ""),
148             "sim_length_m": mapping.get("length_m", ""),
149             "sim_width_mm": mapping.get("width_mm", ""),
150             "kict_image_file": kict.get("image_file", ""),
151             "kict_mask_file": kict.get("mask_file", ""),
152             "kict_image_path": normalize_kict_path(kict, "image_file", "image_path"),
153             "kict_mask_path": normalize_kict_path(kict, "mask_file", "mask_path"),
154             "kict_area_px": kict.get("area_px", ""),
155             "kict_bbox_x1": kict.get("bbox_x1", ""),
156             "kict_bbox_y1": kict.get("bbox_y1", ""),
157             "kict_bbox_x2": kict.get("bbox_x2", ""),
158             "kict_bbox_y2": kict.get("bbox_y2", ""),
159             "kict_center_x": kict.get("center_x", ""),
160             "kict_center_y": kict.get("center_y", ""),
161             "kict_mask_width": kict.get("mask_width", ""),
162             "kict_mask_height": kict.get("mask_height", ""),
163             "has_crack": "True",
164             # Cyclic KICT assignment is demo evidence, not repeated physical observation.
165             "observation_source": "kict_static_mask_cyclic_demo",
166             "comparability_status": "not_longitudinally_comparable",
167         }
```



```
168         )
169     return output_rows, missing_inspection_count, missing_image_ids
170
171
172 def validate_records(rows: list[dict[str, str]]) -> None:
173     if not rows:
174         raise ValueError("robot_kict_frame_records.csv would be empty")
175     for row in rows:
176         if not row["disease_id"]:
177             raise ValueError("Merged row missing disease_id")
178         for key in ["timestamp", "mileage_m", "mileage_text", "ring_id", "clock_direction"]:
179             if row.get(key, "") == "":
180                 raise ValueError(f"Merged row missing engineering metadata field: {key}")
181         if not row["kict_image_path"] or not row["kict_mask_path"]:
182             raise ValueError("Merged row missing KICT image/mask path")
183         if int_value(row, "kict_area_px") <= 0:
184             raise ValueError("Merged row has non-positive kict_area_px")
185         for key in ["kict_bbox_x1", "kict_bbox_y1", "kict_bbox_x2", "kict_bbox_y2", "kict_center_x", "kict_center_y"]:
186             if row.get(key, "") == "":
187                 raise ValueError(f"Merged row missing {key}")
188
189
190 def write_csv(rows: list[dict[str, str]], output_csv: Path) -> None:
191     output_csv.parent.mkdir(parents=True, exist_ok=True)
192     with output_csv.open("w", encoding="utf-8-sig", newline="") as file:
193         writer = csv.DictWriter(file, fieldnames=OUTPUT_COLUMNS)
194         writer.writeheader()
195         writer.writerows(rows)
196
197
198 def write_report(
199     report_file: Path,
200     paths: dict[str, Path],
201     inspection_rows: list[dict[str, str]],
202     mapping_rows: list[dict[str, str]],
203     kict_rows: list[dict[str, str]],
204     valid_kict_rows: list[dict[str, str]],
205     output_rows: list[dict[str, str]],
206     missing_inspection_count: int,
207 ) -> None:
208     disease_type_counts = Counter(row["disease_type"] for row in output_rows)
209     inspection_counts = Counter(row["inspection_id"] for row in output_rows)
210     disease_type_lines = "\n".join(f"- {name}: {count}" for name, count in sorted(disease_type_counts.items()))
211     inspection_lines = "\n".join(f"- {name}: {count}" for name, count in sorted(inspection_counts.items()))
212
213     content = f"""# 机器人巡检仿真表与 KICT Mask 特征合并报告
214
215 ## 输入文件
216
217 - inspection_sequence.csv: {paths["inspection_sequence"]}
```

```
218 - frame_disease_mapping.csv: {paths["frame_disease_mapping"]}
219 - kict_mask_features.csv: {paths["kict_mask_features"]}
220
221 ## 行数统计
222
223 - inspection_sequence.csv 行数: {len(inspection_rows)}
224 - frame_disease_mapping.csv 行数: {len(mapping_rows)}
225 - kict_mask_features.csv 行数: {len(kict_rows)}
226 - 有效 KICT 裂缝样本数: {len(valid_kict_rows)}
227 - robot_kict_frame_records.csv 行数: {len(output_rows)}
228 - disease_id 数量: {len({row["disease_id"] for row in output_rows})}
229 - 无法匹配 inspection_sequence 的 image_id 数量: {missing_inspection_count}
230
231 ## disease_type 分布
232
233 {disease_type_lines}
234
235 ## 巡检次数统计
236
237 {inspection_lines}
238
239 ## 说明
240
241 本阶段将 KICT 数据集中的真实裂缝图像、mask 路径及几何特征接入机器人隧道巡检仿真表。最终生成的 robot_kict_frame_records.csv 同时包含仿真的时间、里程、环号、方位
    、病害编号，以及真实 mask 计算得到的裂缝面积、bbox 和中心点，可用于后续工程化描述生成、时空聚合和变化监测。
242 """
243     report_file.parent.mkdir(parents=True, exist_ok=True)
244     report_file.write_text(content, encoding="utf-8")
245
246
247 def main() -> None:
248     args = parse_args()
249     allow_fallback = args.allow_default_fallback
250     paths = {
251         "inspection_sequence": resolve_input_file(
252             args.sim_dir,
253             "inspection_sequence.csv",
254             allow_fallback=allow_fallback,
255         ),
256         "frame_disease_mapping": resolve_input_file(
257             args.sim_dir,
258             "frame_disease_mapping.csv",
259             allow_fallback=allow_fallback,
260         ),
261         "kict_mask_features": args.kict_features,
262     }
263     inspection_rows = read_csv(paths["inspection_sequence"])
264     mapping_rows = read_csv(paths["frame_disease_mapping"])
265     kict_rows = read_csv(paths["kict_mask_features"])
266     valid_kict_rows = filter_valid_kict_samples(kict_rows)
```

```

267     output_rows, missing_inspection_count, missing_image_ids = build_records(
268         inspection_rows,
269         mapping_rows,
270         valid_kict_rows,
271     )
272     if missing_inspection_count:
273         preview = ", ".join(missing_image_ids[:5])
274         raise ValueError(
275             f"[missing_inspection_count] image_id values were not found in inspection_sequence.csv: {preview}"
276         )
277     validate_records(output_rows)
278     write_csv(output_rows, args.output_csv)
279     write_report(
280         args.report_file,
281         paths,
282         inspection_rows,
283         mapping_rows,
284         kict_rows,
285         valid_kict_rows,
286         output_rows,
287         missing_inspection_count,
288     )
289
290     print("机器人巡检仿真表与 KICT mask 特征合并完成")
291     print(f"inspection_sequence rows: {len(inspection_rows)}")
292     print(f"frame_disease_mapping rows: {len(mapping_rows)}")
293     print(f"valid KICT samples: {len(valid_kict_rows)}")
294     print(f"output rows: {len(output_rows)}")
295     print(f"output file: {args.output_csv}")
296     print(f"report file: {args.report_file}")
297
298
299 if __name__ == "__main__":
300     main()
# --- scripts/generate_engineering_report.py ---
1  from __future__ import annotations
2
3  import argparse
4  import csv
5  from collections import Counter, defaultdict
6  from pathlib import Path
7
8  try:
9      from scripts.report_paths import report_path as display_report_path
10 except ModuleNotFoundError: # Direct ``python scripts/...`` execution.
11     from report_paths import report_path as display_report_path
12
13
14 DEFAULT_INPUT = Path("data/simulated/robot_kict_frame_records.csv")
15 DEFAULT_OUTPUT_CSV = Path("data/simulated/disease_engineering_report.csv")

```

```
16 DEFAULT_MARKDOWN_REPORT = Path("outputs/disease_engineering_report.md")
17 DEFAULT_SUMMARY_REPORT = Path("outputs/disease_engineering_report_summary.md")
18
19 REQUIRED_COLUMNS = [
20     "image_id",
21     "inspection_id",
22     "frame_id",
23     "timestamp",
24     "mileage_m",
25     "mileage_text",
26     "ring_id",
27     "clock_direction",
28     "disease_id",
29     "disease_type",
30     "kict_image_path",
31     "kict_mask_path",
32     "kict_area_px",
33     "kict_bbox_x1",
34     "kict_bbox_y1",
35     "kict_bbox_x2",
36     "kict_bbox_y2",
37     "kict_center_x",
38     "kict_center_y",
39     "has_crack",
40     "observation_source",
41     "comparability_status",
42 ]
43
44 OUTPUT_COLUMNS = [
45     "inspection_id",
46     "disease_id",
47     "disease_type",
48     "frame_count",
49     "start_frame",
50     "end_frame",
51     "start_time",
52     "end_time",
53     "start_mileage_m",
54     "end_mileage_m",
55     "start_mileage_text",
56     "end_mileage_text",
57     "start_ring",
58     "end_ring",
59     "main_clock_direction",
60     "max_area_px",
61     "mean_area_px",
62     "total_area_px",
63     "max_bbox_x1",
64     "max_bbox_y1",
65     "max_bbox_x2",
```

```
66     "max_bbox_y2",
67     "representative_image_path",
68     "representative_mask_path",
69     "risk_level",
70     "observation_source",
71     "comparability_status",
72     "engineering_description",
73 ]
74
75 DISEASE_TYPE_ZH = {
76     "crack": "裂缝",
77     "water_leakage": "渗水",
78     "spalling": "剥落",
79     "unknown": "未知病害",
80 }
81
82
83 def parse_args() -> argparse.Namespace:
84     parser = argparse.ArgumentParser(description="Generate engineering disease reports from robot/KICT frame records.")
85     parser.add_argument("--input-csv", type=Path, default=DEFAULT_INPUT, help="robot_kict_frame_records.csv path.")
86     parser.add_argument("--output-csv", type=Path, default=DEFAULT_OUTPUT_CSV, help="Object-level output CSV path.")
87     parser.add_argument(
88         "--markdown-report",
89         type=Path,
90         default=DEFAULT_MARKDOWN_REPORT,
91         help="Detailed Markdown report path.",
92     )
93     parser.add_argument(
94         "--summary-report",
95         type=Path,
96         default=DEFAULT_SUMMARY_REPORT,
97         help="Summary Markdown report path.",
98     )
99     return parser.parse_args()
100
101
102 def read_input_rows(path: Path) -> list[dict[str, str]]:
103     if not path.exists():
104         raise FileNotFoundError(f"Input file not found: {path}")
105     with path.open(encoding="utf-8-sig", newline="") as file:
106         rows = list(csv.DictReader(file))
107     if not rows:
108         raise ValueError(f"Input table is empty: {path}")
109
110     missing = [column for column in REQUIRED_COLUMNS if column not in rows[0]]
111     if missing:
112         raise ValueError(f"Input table missing required columns: {' '.join(missing)}")
113
114     # 先做输入字段验证，后续聚合就可以直接使用已知结构。
115     for index, row in enumerate(rows, start=2):
```

```
116         parse_int(row["frame_id"], f"frame_id at CSV line {index}")
117         parse_float(row["mileage_m"], f"mileage_m at CSV line {index}")
118         parse_int(row["ring_id"], f"ring_id at CSV line {index}")
119         parse_int(row["kict_area_px"], f"kict_area_px at CSV line {index}")
120     return rows
121
122
123 def parse_int(value: str, label: str) -> int:
124     try:
125         return int(float(value))
126     except (TypeError, ValueError) as exc:
127         raise ValueError(f"{label} must be numeric, got {value!r}") from exc
128
129
130 def parse_float(value: str, label: str) -> float:
131     try:
132         return float(value)
133     except (TypeError, ValueError) as exc:
134         raise ValueError(f"{label} must be numeric, got {value!r}") from exc
135
136
137 def disease_type_zh(disease_type: str) -> str:
138     return DISEASE_TYPE_ZH.get(disease_type, "未知类型病害")
139
140
141 def risk_by_area(max_area_px: int) -> str:
142     if max_area_px < 1500:
143         return "低"
144     if max_area_px < 3000:
145         return "中"
146     return "高"
147
148
149 def range_text(start: str, end: str, same_template: str, range_template: str) -> str:
150     if start == end:
151         return same_template.format(start=start)
152     return range_template.format(start=start, end=end)
153
154
155 def build_description(record: dict[str, str]) -> str:
156     mileage_phrase = range_text(
157         record["start_mileage_text"],
158         record["end_mileage_text"],
159         "在 {start}",
160         "在 {start} 至 {end}",
161     )
162     ring_phrase = range_text(
163         record["start_ring"],
164         record["end_ring"],
165         "第 {start} 环",
```

```

166         "{start} 至 {end} 环",
167     )
168     visible_phrase = "单帧可见" if record["frame_count"] == "1" else f"连续 {record['frame_count']} 帧可见"
169     type_name = disease_type_zh(record["disease_type"])
170     return (
171         f"巡检 {record['inspection_id']} 中, 机器人于 {record['start_time']} 至 {record['end_time']} "
172         f"{mileage_phrase}, {ring_phrase}, {record['main_clock_direction']} 方向发现 {type_name} "
173         f"{record['disease_id']}. 该病害 {visible_phrase}, 最大裂缝面积约为 {record['max_area_px']} px□, "
174         f"平均面积约为 {record['mean_area_px']} px□, 风险等级为 {record['risk_level']}, 建议进行人工复核."
175     )
176
177
178 def aggregate_group(rows: list[dict[str, str]]) -> dict[str, str]:
179     if not rows:
180         raise ValueError("Each inspection_id + disease_id group must contain at least one row")
181
182     rows_by_frame = sorted(rows, key=lambda row: parse_int(row["frame_id"], "frame_id"))
183     rows_by_time = sorted(rows, key=lambda row: row["timestamp"])
184     rows_by_mileage = sorted(rows, key=lambda row: parse_float(row["mileage_m"], "mileage_m"))
185     rows_by_area = sorted(rows, key=lambda row: parse_int(row["kict_area_px"], "kict_area_px"), reverse=True)
186     representative = rows_by_area[0]
187     areas = [parse_int(row["kict_area_px"], "kict_area_px") for row in rows]
188     max_area = max(areas)
189     mean_area = round(sum(areas) / len(areas), 2)
190     total_area = sum(areas)
191     clock_counts = Counter(row["clock_direction"] for row in rows)
192     main_clock_direction = clock_counts.most_common(1)[0][0]
193     # Direct helper callers may carry legacy rows; degrade safely instead of inventing comparability.
194     sources = sorted([row.get("observation_source") or "legacy_unverified_source" for row in rows])
195     statuses = {row.get("comparability_status") or "not_longitudinally_comparable" for row in rows}
196     # Directional conclusions require explicit verified comparability for every frame.
197     comparability_status = "verified_comparable" if statuses == {"verified_comparable"} else "not_longitudinally_comparable"
198
199     record = {
200         "inspection_id": rows[0]["inspection_id"],
201         "disease_id": rows[0]["disease_id"],
202         "disease_type": rows[0]["disease_type"],
203         "frame_count": str(len(rows)),
204         "start_frame": rows_by_frame[0]["frame_id"],
205         "end_frame": rows_by_frame[-1]["frame_id"],
206         "start_time": rows_by_time[0]["timestamp"],
207         "end_time": rows_by_time[-1]["timestamp"],
208         "start_mileage_m": rows_by_mileage[0]["mileage_m"],
209         "end_mileage_m": rows_by_mileage[-1]["mileage_m"],
210         "start_mileage_text": rows_by_mileage[0]["mileage_text"],
211         "end_mileage_text": rows_by_mileage[-1]["mileage_text"],
212         "start_ring": str(min(parse_int(row["ring_id"], "ring_id") for row in rows)),
213         "end_ring": str(max(parse_int(row["ring_id"], "ring_id") for row in rows)),
214         "main_clock_direction": main_clock_direction,
215         "max_area_px": str(max_area),

```

```
216         "mean_area_px": str(mean_area),
217         "total_area_px": str(total_area),
218         "max_bbox_x1": representative["kict_bbox_x1"],
219         "max_bbox_y1": representative["kict_bbox_y1"],
220         "max_bbox_x2": representative["kict_bbox_x2"],
221         "max_bbox_y2": representative["kict_bbox_y2"],
222         "representative_image_path": representative["kict_image_path"],
223         "representative_mask_path": representative["kict_mask_path"],
224         "risk_level": risk_by_area(max_area),
225         "observation_source": sources[0] if len(sources) == 1 else "mixed_sources",
226         "comparability_status": comparability_status,
227     }
228     record["engineering_description"] = build_description(record)
229     return record
230
231
232 def aggregate_rows(rows: list[dict[str, str]]) -> list[dict[str, str]]:
233     groups: dict[tuple[str, str], list[dict[str, str]]] = defaultdict(list)
234     for row in rows:
235         groups[(row["inspection_id"], row["disease_id"])].append(row)
236
237     records = [aggregate_group(group_rows) for _, group_rows in sorted(groups.items())]
238     if not records:
239         raise ValueError("Output CSV would be empty")
240     if any(not record["engineering_description"] for record in records):
241         raise ValueError("engineering_description should not be empty")
242     return records
243
244
245 def write_csv_report(records: list[dict[str, str]], output_csv: Path) -> None:
246     output_csv.parent.mkdir(parents=True, exist_ok=True)
247     with output_csv.open("w", encoding="utf-8-sig", newline="") as file:
248         writer = csv.DictWriter(file, fieldnames=OUTPUT_COLUMNS)
249         writer.writeheader()
250         writer.writerows(records)
251
252
253 def write_markdown_report(records: list[dict[str, str]], input_csv: Path, report_path: Path) -> None:
254     report_path.parent.mkdir(parents=True, exist_ok=True)
255     records_by_inspection: dict[str, list[dict[str, str]]] = defaultdict(list)
256     for record in records:
257         records_by_inspection[record["inspection_id"]].append(record)
258
259     lines = [
260         "# 机器人隧道巡检工程化病害报告",
261         "",
262         f"本报告来自: `{display_report_path(input_csv)}`",
263         "",
264     ]
265     for inspection_id, inspection_records in sorted(records_by_inspection.items()):
```



```

266         lines.extend([f"## 巡检 {inspection_id}", ""])
267     for record in inspection_records:
268         type_name = disease_type_zh(record["disease_type"])
269         lines.extend(
270             [
271                 f"### {record['disease_id']} - {type_name}",
272                 "",
273                 f"- 里程范围: {record['start_mileage_text']} 至 {record['end_mileage_text']}",
274                 f"- 环号范围: {record['start_ring']} 至 {record['end_ring']} 环",
275                 f"- 方位: {record['main_clock_direction']} 方向",
276                 f"- 可见帧数: {record['frame_count']}",
277                 f"- 最大面积: {record['max_area_px']} px",
278                 f"- 平均面积: {record['mean_area_px']} px",
279                 f"- 风险等级: {record['risk_level']}",
280                 f"- 代表图像: {record['representative_image_path']}",
281                 f"- 代表 mask: {record['representative_mask_path']}",
282                 "",
283                 "工程描述: ",
284                 record["engineering_description"],
285                 "",
286                 "-----",
287                 "",
288             ]
289         )
290     report_path.write_text("\n".join(lines), encoding="utf-8")
291
292
293 def write_summary_report(
294     input_csv: Path,
295     output_csv: Path,
296     markdown_report: Path,
297     summary_report: Path,
298     input_rows: list[dict[str, str]],
299     records: list[dict[str, str]],
300 ) -> None:
301     summary_report.parent.mkdir(parents=True, exist_ok=True)
302     inspection_counts = Counter(record["inspection_id"] for record in records)
303     risk_counts = Counter(record["risk_level"] for record in records)
304     type_counts = Counter(record["disease_type"] for record in records)
305     inspection_lines = "\n".join(f"- {name}: {count}" for name, count in sorted(inspection_counts.items()))
306     risk_lines = "\n".join(f"- {name}: {count}" for name, count in sorted(risk_counts.items()))
307     type_lines = "\n".join(f"- {disease_type_zh(name)} ({name}): {count}" for name, count in sorted(type_counts.items()))
308
309     content = f"""# 工程化病害报告摘要
310
311 ## 输入文件
312
313 - {display_report_path(input_csv)}
314
315 ## 输出文件

```

```
316
317 - {display_report_path(output_csv)}
318 - {display_report_path(markdown_report)}
319 - {display_report_path(summary_report)}
320
321 ## 统计信息
322
323 - 总记录行数: {len(input_rows)}
324 - 聚合后的病害对象数量: {len(records)}
325 - 巡检次数: {len(inspection_counts)}
326
327 ## 每次巡检的病害对象数量
328
329 {inspection_lines}
330
331 ## 风险等级分布
332
333 {risk_lines}
334
335 ## 病害类型分布
336
337 {type_lines}
338
339 ## 说明
340
341 本阶段基于 robot_kict_frame_records.csv, 将连续帧中的同一 disease_id 聚合为工程化病害对象, 并生成病害对象级 CSV 与 Markdown 报告。该结果可用于后续时空聚合验证、
    静态面积审计、可比性检查和工程化文本输出。
342 """
343     summary_report.write_text(content, encoding="utf-8")
344
345
346 def main() -> None:
347     args = parse_args()
348     input_rows = read_input_rows(args.input_csv)
349     records = aggregate_rows(input_rows)
350     write_csv_report(records, args.output_csv)
351     write_markdown_report(records, args.input_csv, args.markdown_report)
352     write_summary_report(args.input_csv, args.output_csv, args.markdown_report, args.summary_report, input_rows, records)
353
354     # 后续可基于 disease_engineering_report.csv 继续做跨巡检变化分析、面积增长率、
355     # 长宽/骨架变化监测、风险趋势判断、简单时间序列预测, 以及 dashboard 或报告导出。
356     print("工程化病害报告生成完成")
357     print(f"input rows: {len(input_rows)}")
358     print(f"aggregated disease records: {len(records)}")
359     print(f"output csv: {args.output_csv}")
360     print(f"markdown report: {args.markdown_report}")
361     print(f"summary report: {args.summary_report}")
362
363
364 if __name__ == "__main__":
```

```
365     main()
# --- scripts/analyze_disease_growth.py ---
1   from __future__ import annotations
2
3   import argparse
4   import csv
5   import math
6   from collections import Counter, defaultdict
7   from pathlib import Path
8
9   try:
10      from scripts.report_paths import report_path as display_report_path
11  except ModuleNotFoundError: # Direct ``python scripts/...`` execution.
12      from report_paths import report_path as display_report_path
13
14
15  DEFAULT_INPUT = Path("data/simulated/disease_engineering_report.csv")
16  DEFAULT_OUTPUT_CSV = Path("data/simulated/disease_growth_analysis.csv")
17  DEFAULT_MARKDOWN_REPORT = Path("outputs/disease_growth_analysis_report.md")
18  DEFAULT_SUMMARY_REPORT = Path("outputs/disease_growth_analysis_summary.md")
19
20  REQUIRED_COLUMNS = [
21      "inspection_id",
22      "disease_id",
23      "disease_type",
24      "frame_count",
25      "start_frame",
26      "end_frame",
27      "start_time",
28      "end_time",
29      "start_mileage_m",
30      "end_mileage_m",
31      "start_mileage_text",
32      "end_mileage_text",
33      "start_ring",
34      "end_ring",
35      "main_clock_direction",
36      "max_area_px",
37      "mean_area_px",
38      "total_area_px",
39      "risk_level",
40      "observation_source",
41      "comparability_status",
42      "engineering_description",
43  ]
44
45  OUTPUT_COLUMNS = [
46      "disease_id",
47      "disease_type",
48      "inspection_count",
```

```
49     "first_inspection",
50     "last_inspection",
51     "first_area_px",
52     "last_area_px",
53     "area_growth_px",
54     "area_growth_rate",
55     "first_mean_area_px",
56     "last_mean_area_px",
57     "mean_area_growth_rate",
58     "first_frame_count",
59     "last_frame_count",
60     "frame_count_change",
61     "first_risk_level",
62     "last_risk_level",
63     "risk_level_change",
64     "growth_trend",
65     "attention_level",
66     "measurement_basis",
67     "claim_level",
68     "comparability_status",
69     "first_mileage_range",
70     "last_mileage_range",
71     "main_clock_direction",
72     "growth_description",
73 ]
74
75 DISEASE_TYPE_ZH = {
76     "crack": "裂缝",
77     "water_leakage": "渗水",
78     "spalling": "剥落",
79     "unknown": "未知病害",
80 }
81 RISK_SCORE = {"低": 1, "中": 2, "高": 3}
82 ATTENTION_ORDER = ["重点关注", "持续观察", "常规记录", "待补充巡检"]
83
84
85 def parse_args() -> argparse.Namespace:
86     parser = argparse.ArgumentParser(description="Analyze disease growth across robot inspections.")
87     parser.add_argument("--input-csv", type=Path, default=DEFAULT_INPUT, help="disease_engineering_report.csv path.")
88     parser.add_argument("--output-csv", type=Path, default=DEFAULT_OUTPUT_CSV, help="Growth analysis CSV path.")
89     parser.add_argument(
90         "--markdown-report",
91         type=Path,
92         default=DEFAULT_MARKDOWN_REPORT,
93         help="Detailed growth Markdown report path.",
94     )
95     parser.add_argument(
96         "--summary-report",
97         type=Path,
98         default=DEFAULT_SUMMARY_REPORT,
```

```
99         help="Summary Markdown report path.",
100     )
101     return parser.parse_args()
102
103
104 def parse_int(value: str, label: str) -> int:
105     try:
106         return int(float(value))
107     except (TypeError, ValueError) as exc:
108         raise ValueError(f"{label} must be numeric, got {value!r}") from exc
109
110
111 def parse_float(value: str, label: str) -> float:
112     try:
113         return float(value)
114     except (TypeError, ValueError) as exc:
115         raise ValueError(f"{label} must be numeric, got {value!r}") from exc
116
117
118 def read_input_rows(path: Path) -> list[dict[str, str]]:
119     if not path.exists():
120         raise FileNotFoundError(f"Input file not found: {path}")
121     with path.open(encoding="utf-8-sig", newline="") as file:
122         rows = list(csv.DictReader(file))
123     if not rows:
124         raise ValueError(f"Input table is empty: {path}")
125
126     missing = [column for column in REQUIRED_COLUMNS if column not in rows[0]]
127     if missing:
128         raise ValueError(f"Input table missing required columns: {' '.join(missing)}")
129
130     # 提前验证核心数值字段，避免后续报告里出现隐性脏数据。
131     for index, row in enumerate(rows, start=2):
132         parse_int(row["max_area_px"], f"max_area_px at CSV line {index}")
133         parse_float(row["mean_area_px"], f"mean_area_px at CSV line {index}")
134         parse_int(row["frame_count"], f"frame_count at CSV line {index}")
135     return rows
136
137
138 def inspection_sort_key(inspection_id: str) -> tuple[int, str]:
139     digits = "".join(char for char in inspection_id if char.isdigit())
140     return (int(digits) if digits else 0, inspection_id)
141
142
143 def disease_type_zh(disease_type: str) -> str:
144     return DISEASE_TYPE_ZH.get(disease_type, "未知类型病害")
145
146
147 def mileage_range(row: dict[str, str]) -> str:
148     if row["start_mileage_text"] == row["end_mileage_text"]:
```

```
149         return row["start_mileage_text"]
150     return f"[row['start_mileage_text']] 至 [row['end_mileage_text']]"
151
152
153 def safe_growth_rate(delta: float, base: float) -> float:
154     if base == 0:
155         return 0.0
156     return round(delta / base, 4)
157
158
159 def risk_level_change(first_risk: str, last_risk: str) -> int:
160     return RISK_SCORE.get(last_risk, 0) - RISK_SCORE.get(first_risk, 0)
161
162
163 def growth_trend(inspection_count: int, area_growth_rate: float, risk_change: int) -> str:
164     if inspection_count == 1:
165         return "数据不足"
166     if area_growth_rate >= 0.5 or risk_change >= 1:
167         return "明显增长"
168     if 0.15 <= area_growth_rate < 0.5:
169         return "轻微增长"
170     if -0.15 < area_growth_rate < 0.15:
171         return "基本稳定"
172     if area_growth_rate <= -0.15:
173         return "面积减小"
174     return "基本稳定"
175
176
177 def attention_level(last_risk_level: str, trend: str) -> str:
178     # 单次巡检无法比较增长趋势, 优先归入待补充巡检。
179     if trend == "数据不足":
180         return "待补充巡检"
181     if last_risk_level == "高" or trend == "明显增长":
182         return "重点关注"
183     if last_risk_level == "中" or trend == "轻微增长":
184         return "持续观察"
185     if last_risk_level == "低" and trend == "基本稳定":
186         return "常规记录"
187     return "持续观察"
188
189
190 def build_growth_description(record: dict[str, str]) -> str:
191     type_name = disease_type_zh(record["disease_type"])
192     if record["inspection_count"] == "1":
193         return (
194             f"病害 {record['disease_id']} 为{type_name}, 目前仅在 {record['first_inspection']} 中出现, "
195             f"位于 {record['first_mileage_range']}, {record['main_clock_direction']} 方向。"
196             "由于缺少跨巡检对比数据, 当前仅作为基线记录, 建议后续巡检继续跟踪。"
197         )
198
```

```
199     if record["comparability_status"] != "verified_comparable":
200         return (
201             f"病害 {record['disease_id']} 为{type_name}, 主要位于 {record['last_mileage_range']}, "
202             f"{record['main_clock_direction']}方向。该病害在 {record['inspection_count']} 次巡检中保留了静态 mask 面积证据, "
203             f"首次/末次面积分别为 {record['first_area_px']} px□ 和 {record['last_area_px']} px□。"
204             "由于观测来自不可纵向比较的静态 KICT 样本, 面积差仅用于审计, 不作跨巡检方向判断。"
205         )
206
207     area_growth_percent = round(float(record["area_growth_rate"]) * 100, 1)
208     return (
209         f"病害 {record['disease_id']} 为{type_name}, 主要位于 {record['last_mileage_range']}, "
210         f"{record['main_clock_direction']}方向。该病害共出现在 {record['inspection_count']} 次巡检中, "
211         f"首次巡检 {record['first_inspection']} 最大面积约为 {record['first_area_px']} px□, "
212         f"末次巡检 {record['last_inspection']} 最大面积约为 {record['last_area_px']} px□, "
213         f"面积变化 {record['area_growth_px']} px□, 增长率约为 {area_growth_percent}%。"
214         f"风险等级由{record['first_risk_level']}变为{record['last_risk_level']}, "
215         f"规则证据提示为{record['growth_trend']}, 关注等级为{record['attention_level']}。"
216     )
217
218
219 def most_common_clock(rows: list[dict[str, str]]) -> str:
220     return Counter(row["main_clock_direction"] for row in rows).most_common(1)[0][0]
221
222
223 def aggregate_disease(rows: list[dict[str, str]]) -> dict[str, str]:
224     if not rows:
225         raise ValueError("Each disease_id group must contain at least one row")
226     ordered = sorted(rows, key=lambda row: inspection_sort_key(row["inspection_id"]))
227     first = ordered[0]
228     last = ordered[-1]
229
230     first_area = parse_int(first["max_area_px"], "first max_area_px")
231     last_area = parse_int(last["max_area_px"], "last max_area_px")
232     area_growth = last_area - first_area
233     area_rate = safe_growth_rate(area_growth, first_area)
234     first_mean = parse_float(first["mean_area_px"], "first mean_area_px")
235     last_mean = parse_float(last["mean_area_px"], "last mean_area_px")
236     mean_rate = safe_growth_rate(last_mean - first_mean, first_mean)
237     first_frames = parse_int(first["frame_count"], "first frame_count")
238     last_frames = parse_int(last["frame_count"], "last frame_count")
239     risk_change = risk_level_change(first["risk_level"], last["risk_level"])
240     all_verified_comparable = all(row.get("comparability_status") == "verified_comparable" for row in ordered)
241     trend = growth_trend(len(ordered), area_rate, risk_change)
242     attention = attention_level(last["risk_level"], trend)
243     if len(ordered) == 1:
244         measurement_basis = "single_inspection_area_rule"
245         claim_level = "baseline_only"
246         comparability_status = "insufficient_history"
247     elif not all_verified_comparable:
248         trend = "不可比较"
```

```

249         attention = attention_level(last["risk_level"], "")
250         measurement_basis = "static_mask_area_descriptive_only"
251         claim_level = "rule_evidence_only"
252         comparability_status = "not_longitudinally_comparable"
253     else:
254         measurement_basis = "cross_inspection_area_rule"
255         claim_level = "rule_evidence_only"
256         comparability_status = "verified_comparable"
257
258     record = {
259         "disease_id": first["disease_id"],
260         "disease_type": first["disease_type"],
261         "inspection_count": str(len(ordered)),
262         "first_inspection": first["inspection_id"],
263         "last_inspection": last["inspection_id"],
264         "first_area_px": str(first_area),
265         "last_area_px": str(last_area),
266         "area_growth_px": str(area_growth),
267         "area_growth_rate": str(area_rate),
268         "first_mean_area_px": str(first_mean),
269         "last_mean_area_px": str(last_mean),
270         "mean_area_growth_rate": str(mean_rate),
271         "first_frame_count": str(first_frames),
272         "last_frame_count": str(last_frames),
273         "frame_count_change": str(last_frames - first_frames),
274         "first_risk_level": first["risk_level"],
275         "last_risk_level": last["risk_level"],
276         "risk_level_change": str(risk_change),
277         "growth_trend": trend,
278         "attention_level": attention,
279         "measurement_basis": measurement_basis,
280         "claim_level": claim_level,
281         "comparability_status": comparability_status,
282         "first_mileage_range": mileage_range(first),
283         "last_mileage_range": mileage_range(last),
284         "main_clock_direction": most_common_clock(ordered),
285     }
286     record["growth_description"] = build_growth_description(record)
287     return record
288
289
290 def aggregate_rows(rows: list[dict[str, str]]) -> list[dict[str, str]]:
291     groups: dict[str, list[dict[str, str]]] = defaultdict(list)
292     for row in rows:
293         groups[row["disease_id"]].append(row)
294
295     records = [aggregate_disease(group_rows) for _, group_rows in sorted(groups.items())]
296     if not records:
297         raise ValueError("Output CSV would be empty")
298     for record in records:

```



```
299         if not record["growth_description"]:
300             raise ValueError("growth_description should not be empty")
301         if not math.isfinite(float(record["area_growth_rate"])) or not math.isfinite(
302             float(record["mean_area_growth_rate"]))
303     ):
304         raise ValueError("Growth rates should not be NaN or inf")
305     return records
306
307
308 def write_csv_report(records: list[dict[str, str]], output_csv: Path) -> None:
309     output_csv.parent.mkdir(parents=True, exist_ok=True)
310     with output_csv.open("w", encoding="utf-8-sig", newline="") as file:
311         writer = csv.DictWriter(file, fieldnames=OUTPUT_COLUMNS)
312         writer.writeheader()
313         writer.writerows(records)
314
315
316 def percent_text(rate: str) -> str:
317     return f"{round(float(rate) * 100, 1)}%"
318
319
320 def is_longitudinally_comparable(record: dict[str, str]) -> bool:
321     """Allow directional claims only for explicitly verified observations."""
322
323     return record.get("comparability_status") == "verified_comparable"
324
325
326 def write_markdown_report(
327     records: list[dict[str, str]],
328     input_rows: list[dict[str, str]],
329     input_csv: Path,
330     report_path: Path,
331 ) -> None:
332     report_path.parent.mkdir(parents=True, exist_ok=True)
333     attention_counts = Counter(record["attention_level"] for record in records)
334     inspection_ids = {row["inspection_id"] for row in input_rows}
335     lines = [
336         "# 机器人隧道巡检病害面积审计与可比性报告",
337         "",
338         f"数据来源: `{display_report_path(input_csv)}`",
339         "",
340         "## 总体概况",
341         "",
342         f"- 分析病害数量: {len(records)}",
343         f"- 涉及巡检次数: {len(inspection_ids)}",
344         f"- 重点关注病害数量: {attention_counts.get('重点关注', 0)}",
345         f"- 持续观察病害数量: {attention_counts.get('持续观察', 0)}",
346         f"- 常规记录病害数量: {attention_counts.get('常规记录', 0)}",
347         f"- 待补充巡检病害数量: {attention_counts.get('待补充巡检', 0)}",
348         "",
```

```
349     ]
350
351     records_by_attention: dict[str, list[dict[str, str]]] = defaultdict(list)
352     for record in records:
353         records_by_attention[record["attention_level"]].append(record)
354
355     for level in ATTENTION_ORDER:
356         lines.extend([f"## {level}病害", ""])
357         for record in records_by_attention.get(level, []):
358             type_name = disease_type_zh(record["disease_type"])
359             comparable = is_longitudinally_comparable(record)
360             detail_lines = [
361                 f"### {record['disease_id']} - {type_name}",
362                 "",
363                 f"- 首次巡检: {record['first_inspection']}",
364                 f"- 末次巡检: {record['last_inspection']}",
365             ]
366             if comparable:
367                 detail_lines.extend(
368                     [
369                         f"- 面积变化: {record['first_area_px']} px → {record['last_area_px']} px",
370                         f"- 面积相对差: {percent_text(record['area_growth_rate'])}",
371                         f"- 规则状态: {record['growth_trend']}",
372                     ]
373                 )
374             else:
375                 detail_lines.extend(
376                     [
377                         f"- 静态面积审计: {record['first_area_px']} px → {record['last_area_px']} px",
378                         f"- 描述性面积相对差: {percent_text(record['area_growth_rate'])}",
379                         "- 纵向可比性: 不可比较",
380                         "- 说明: 当前记录不具备纵向比较条件; 该数值仅用于静态样本的描述性审计, 不构成方向性变化或长期变化判断。",
381                     ]
382                 )
383             risk_line = (
384                 f"- 风险变化: {record['first_risk_level']} → {record['last_risk_level']}"
385                 if comparable
386                 else f"- 当前风险等级: {record['last_risk_level']}"
387             )
388             detail_lines.extend(
389                 [
390                     risk_line,
391                     f"- 关注等级: {record['attention_level']}",
392                     f"- 证据等级: {record['claim_level']}",
393                     f"- 可比性状态: {record['comparability_status']}",
394                     "",
395                     "分析描述: ",
396                     record["growth_description"],
397                     "",
398                     "-----",
```

```
399         "",
400     ]
401 )
402     lines.extend(detail_lines)
403     report_path.write_text("\n".join(lines), encoding="utf-8")
404
405
406 def write_summary_report(
407     input_csv: Path,
408     output_csv: Path,
409     markdown_report: Path,
410     summary_report: Path,
411     input_rows: list[dict[str, str]],
412     records: list[dict[str, str]],
413 ) -> None:
414     summary_report.parent.mkdir(parents=True, exist_ok=True)
415     inspection_ids = {row["inspection_id"] for row in input_rows}
416     trend_counts = Counter(record["growth_trend"] for record in records)
417     attention_counts = Counter(record["attention_level"] for record in records)
418     comparable_records = [record for record in records if is_longitudinally_comparable(record)]
419     risk_change_counts = Counter(record["risk_level_change"] for record in comparable_records)
420     trend_lines = "\n".join(f"- {name}: {count}" for name, count in sorted(trend_counts.items()))
421     attention_lines = "\n".join(f"- {name}: {count}" for name, count in sorted(attention_counts.items()))
422     risk_change_lines = (
423         "\n".join(f"- {name}: {count}" for name, count in sorted(risk_change_counts.items()))
424         or "- 暂无可比数据"
425     )
426
427     content = f"""# 病害面积审计与可比性摘要
428
429 ## 输入文件
430
431 - {display_report_path(input_csv)}
432
433 ## 输出文件
434
435 - {display_report_path(output_csv)}
436 - {display_report_path(markdown_report)}
437 - {display_report_path(summary_report)}
438
439 ## 统计信息
440
441 - 输入记录行数: {len(input_rows)}
442 - 输出病害数量: {len(records)}
443 - 巡检次数: {len(inspection_ids)}
444
445 ## 可比性与规则状态分布
446
447 {trend_lines}
448
```

```
449 ## 关注等级分布
450
451 {attention_lines}
452
453 ## 已验证可比记录风险等级变化统计
454
455 {risk_change_lines}
456
457 ## 说明
458
459 本阶段基于 disease_engineering_report.csv, 对同一 disease_id 在多次巡检中的面积、可见帧数和风险等级进行规则比较, 生成工程复检提示。对不可纵向比较记录, 面积数值
    仅作描述性审计, 不构成变化方向判断。该结果不等于真实结构安全结论, 后续应接入真实连续巡检数据和人工复核。
460 """
461     summary_report.write_text(content, encoding="utf-8")
462
463
464 def main() -> None:
465     args = parse_args()
466     input_rows = read_input_rows(args.input_csv)
467     records = aggregate_rows(input_rows)
468     write_csv_report(records, args.output_csv)
469     write_markdown_report(records, input_rows, args.input_csv, args.markdown_report)
470     write_summary_report(args.input_csv, args.output_csv, args.markdown_report, args.summary_report, input_rows, records)
471
472     # 后续可基于 disease_growth_analysis.csv 做趋势预测、增长曲线可视化、
473     # 按里程段统计风险、生成重点复检清单、接入 dashboard、导出 Word/PDF 报告。
474     print("病害面积审计与可比性分析完成")
475     print(f"input rows: {len(input_rows)}")
476     print(f"disease count: {len(records)}")
477     print(f"output csv: {args.output_csv.as_posix()}")
478     print(f"markdown report: {args.markdown_report.as_posix()}")
479     print(f"summary report: {args.summary_report.as_posix()}")
480
481
482 if __name__ == "__main__":
483     main()
# --- robot_sequence.py ---
1 from __future__ import annotations
2
3 import json
4 import re
5 from dataclasses import asdict, dataclass, field
6 from datetime import datetime, timezone
7 from pathlib import Path
8 from typing import Any, Iterable
9
10
11 _MILEAGE_RE = re.compile(r"[Kk]? (?P<km>\d+)\s*\+\s*(?P<m>\d+ (?:\. \d+)?)")
12
13
```

```
14 @dataclass(frozen=True)
15 class FrameRecord:
16     frame_id: str
17     manifest_index: int
18     image_path: str | None = None
19     report_path: str | None = None
20     timestamp: str | None = None
21     mileage: Any = None
22     ring_id: str | None = None
23     camera_id: str | None = None
24     inspection_run_id: str | None = None
25     pose: dict[str, Any] | None = None
26     calibration: dict[str, Any] | None = None
27     depth: Any = None
28     metadata: dict[str, Any] = field(default_factory=dict)
29     ordering_confidence: str = "low"
30     location_confidence: str = "low"
31     limitations: list[str] = field(default_factory=list)
32
33     def to_dict(self) -> dict[str, Any]:
34         return asdict(self)
35
36
37 def _read_manifest(manifest: str | Path | dict[str, Any]) -> tuple[dict[str, Any], Path | None]:
38     if isinstance(manifest, (str, Path)):
39         path = Path(manifest)
40         return json.loads(path.read_text(encoding="utf-8")), path.parent
41     return dict(manifest), None
42
43
44 def _parse_timestamp(value: Any) -> datetime | None:
45     if value in (None, ""):
46         return None
47     if isinstance(value, datetime):
48         return value
49     text = str(value).strip()
50     if not text:
51         return None
52     if text.endswith("Z"):
53         text = f"{text[:-1]}+00:00"
54     try:
55         parsed = datetime.fromisoformat(text)
56     except ValueError:
57         return None
58     if parsed.tzinfo is None:
59         parsed = parsed.replace(tzinfo=timezone.utc)
60     return parsed
61
62
63 def parse_mileage(value: Any) -> float | None:
```

```
64     if value in (None, ""):
65         return None
66     if isinstance(value, (int, float)):
67         return float(value)
68     text = str(value).strip()
69     if not text:
70         return None
71     try:
72         return float(text)
73     except ValueError:
74         pass
75     match = _MILEAGE_RE.search(text)
76     if not match:
77         return None
78     return float(match.group("km")) * 1000.0 + float(match.group("m"))
79
80
81 def _path_text(value: Any, base_dir: Path | None) -> str | None:
82     if value in (None, ""):
83         return None
84     path = Path(str(value))
85     if base_dir is not None and not path.is_absolute():
86         path = base_dir / path
87     return str(path).replace("\\", "/")
88
89
90 def _frame_limitations(frame: dict[str, Any]) -> list[str]:
91     limitations: list[str] = []
92     if parse_mileage(frame.get("mileage")) is None:
93         limitations.append("mileage_missing")
94     if not frame.get("ring_id"):
95         limitations.append("ring_id_missing")
96     if not frame.get("timestamp"):
97         limitations.append("timestamp_missing")
98     if not frame.get("camera_id"):
99         limitations.append("camera_id_missing")
100    if not frame.get("pose"):
101        limitations.append("pose_missing")
102    if not frame.get("calibration"):
103        limitations.append("calibration_missing")
104    if frame.get("depth") in (None, ""):
105        limitations.append("depth_missing")
106    return sorted(set(limitations))
107
108
109 def _confidence(frame: dict[str, Any]) -> tuple[str, str]:
110     has_mileage = parse_mileage(frame.get("mileage")) is not None
111     has_timestamp = _parse_timestamp(frame.get("timestamp")) is not None
112     has_ring = bool(frame.get("ring_id"))
113     ordering = "high" if has_mileage and has_timestamp else "medium" if has_mileage or has_timestamp else "low"
```

```
1434         return
1435     for decision in validated_claims["claim_decision"]["record_decisions"]:
1436         capabilities = decision.get("capabilities", {})
1437         if any(
1438             capabilities.get(name) != "blocked"
1439             for name in (
1440                 "descriptive_difference_claim",
1441                 "directional_change_claim",
1442                 "physical_quantity_change_claim",
1443                 "multi_timepoint_pattern_claim",
1444                 "prediction_claim",
1445             )
1446         ):
1447             raise InspectionWorkflowLifecycleError(
1448                 "Legacy simulated ClaimDecision must remain static-audit only"
1449             )
1450
1451
1452 __all__ = [
1453     "InspectionWorkflowLifecycleError",
1454     "LEGACY_FRAME_RECORDS_PATH",
1455     "run_legacy_simulated",
1456     "run_prepared_task",
1457 ]
# --- web_app.py ---
1   from __future__ import annotations
2
3   import argparse
4   import csv
5   import json
6   import mimetypes
7   import os
8   import re
9   import shutil
10  import threading
11  import time
12  import uuid
13  from dataclasses import dataclass
14  from email import policy
15  from email.parser import BytesParser
16  from http import HTTPStatus
17  from http.server import SimpleHTTPRequestHandler, ThreadingHTTPServer
18  from pathlib import Path
19  from urllib.parse import parse_qs, unquote, urlparse
20
21  from PIL import Image
22
23  from run_confidence_risk import (
24      DEFAULT_SEGFORMER_CHECKPOINT,
25      DEFAULT_SEGFORMER_CONFIG,
```

```

26     DEFAULT_SEGFORMER_REPO_ROOT,
27     IMAGE_SUFFIXES,
28     load_model,
29     process_image,
30 )
31 from inspection_report import build_inspection_report
32 from orchestrator.api import dag_payload, run_payload, runs_payload
33 from orchestrator.runs import RunManager
34
35
36 ROOT = Path(__file__).resolve().parent
37 WEB_ROOT = ROOT / "web_demo"
38 WEB_PLATFORM_ROOT = ROOT / "web"
39 OUTPUT_ROOT = ROOT / "experiments" / "web_live"
40 EVIDENCE_FILES = {
41     "test": ROOT / "experiments" / "enhancement_evidence_test_summary.json",
42     "all": ROOT / "experiments" / "enhancement_evidence_all_summary.json",
43 }
44 ROBOT_ROUTE_REPORT_FILE = WEB_ROOT / "assets" / "robot_route_report.json"
45 SIMULATED_ROOT = ROOT / "data" / "simulated"
46 ENGINEERING_REPORT_FILE = SIMULATED_ROOT / "disease_engineering_report.csv"
47 PRIORITY_RECHECK_FILE = SIMULATED_ROOT / "priority_recheck_list.csv"
48 DISEASE_GROWTH_FILE = SIMULATED_ROOT / "disease_growth_analysis.csv"
49 ROBOT_KICT_FRAME_RECORDS_FILE = SIMULATED_ROOT / "robot_kict_frame_records.csv"
50 ORCHESTRATOR_STATE_FILE = ROOT / "orchestrator" / "state" / "run_state.json"
51 VISUALIZATION_ROOT = ROOT / "outputs" / "visualizations"
52 EVIDENCE_OVERLAY_ROOT = ROOT / "outputs" / "evidence_overlays"
53 VIDEO_DATA_ROOT = ROOT / "data" / "videos"
54 VIDEO_FRAME_ROOT = ROOT / "data" / "video_frames"
55 VIDEO_INSPECTION_ROOT = ROOT / "data" / "video_inspection"
56 VIDEO_OUTPUT_ROOT = ROOT / "outputs" / "video_inspection"
57 DEFAULT_VIDEO_ID = "tunnel_demo"
58 VIDEO_MISSING_MESSAGE = "尚未生成该视频分析产物，请先运行 Step 1 / Step 2 / Step 3。"
59 ALGORITHM_EVENTS_FILE = ROOT / "outputs" / "algorithm_visualization" / "algorithm_events.json"
60 ALGORITHM_EVENTS_MISSING_MESSAGE = "尚未生成算法展示事件流，请先运行 python scripts/generate_algorithm_events.py。"
61 ALGORITHM_EVENT_STATUS_VALUES = {"available", "missing", "optional_missing", "error"}
62 KICT_DATASET_ROOTS = [
63     ROOT / "kict_sample",
64     ROOT.parent / "kict_sample",
65     Path.home() / "Downloads" / "Compressed" / "archive" / "kict_sample",
66 ]
67 VISUALIZATION_FILES = {
68     "attention_level": VISUALIZATION_ROOT / "attention_level_distribution.png",
69     "growth_trend": VISUALIZATION_ROOT / "growth_trend_distribution.png",
70     "risk_change": VISUALIZATION_ROOT / "risk_level_change_distribution.png",
71     "top_growth": VISUALIZATION_ROOT / "top10_area_growth_rate.png",
72     "disease_type": VISUALIZATION_ROOT / "disease_type_distribution.png",
73     "mileage_risk": VISUALIZATION_ROOT / "mileage_risk_distribution.png",
74 }
75 VISUALIZATION_TITLES = {

```



```
76     "attention_level": "关注等级分布",
77     "growth_trend": "可比性与变化状态分布",
78     "risk_change": "已验证可比记录风险等级变化",
79     "top_growth": "可比跨巡检面积审计 Top 10",
80     "disease_type": "病害类型分布",
81     "mileage_risk": "里程段风险分布",
82 }
83 MAX_UPLOAD_BYTES = 20 * 1024 * 1024
84 DEFAULT_WEB_PYTHON = Path("<python-environment>/python.exe")
85
86
87 @dataclass(frozen=True)
88 class WebModelConfig:
89     model_source: str = "segformer"
90     python_executable: Path = DEFAULT_WEB_PYTHON
91     segformer_config: Path = DEFAULT_SEGFORMER_CONFIG
92     segformer_checkpoint: Path = DEFAULT_SEGFORMER_CHECKPOINT
93     segformer_repo_root: Path = DEFAULT_SEGFORMER_REPO_ROOT
94     segformer_device: str = "cuda:0"
95
96 _MODEL = None
97 _CONFIG = None
98 _WEB_MODEL_CONFIG = WebModelConfig()
99 _INFER_LOCK = threading.Lock()
100
101
102 def _safe_filename(filename: str) -> str:
103     name = Path(filename or "upload.jpg").name
104     stem = re.sub(r"[^A-Za-z0-9._-]+", "_", Path(name).stem).strip(".") or "upload"
105     suffix = Path(name).suffix.lower()
106     return f"{stem}{suffix}"
107
108
109 def _is_relative_to(path: Path, root: Path) -> bool:
110     try:
111         path.relative_to(root)
112         return True
113     except ValueError:
114         return False
115
116
117 def _strip_prefix(value: str, prefix: str) -> str:
118     if value.startswith(prefix):
119         return value[len(prefix):]
120     return value
121
122
123 def _json_bytes(payload: dict, status: HTTPStatus = HTTPStatus.OK) -> tuple[int, bytes, str]:
124     return int(status), json.dumps(payload, ensure_ascii=False).encode("utf-8"), "application/json; charset=utf-8"
125
```

```
126
127 def _load_evidence_payload() -> dict:
128     summaries = {}
129     missing = []
130     errors = {}
131     for split, path in EVIDENCE_FILES.items():
132         if not path.exists():
133             missing.append(split)
134             continue
135         try:
136             summaries[split] = json.loads(path.read_text(encoding="utf-8"))
137         except (OSError, json.JSONDecodeError) as exc:
138             errors[split] = str(exc)
139     return {
140         "ok": True,
141         "source": "generated-json" if summaries else "fallback",
142         "summaries": summaries,
143         "missing": missing,
144         "errors": errors,
145     }
146
147
148 def _load_robot_route_report() -> dict:
149     if not ROBOT_ROUTE_REPORT_FILE.exists():
150         return {
151             "ok": True,
152             "source": "fallback",
153             "report": None,
154             "missing": str(ROBOT_ROUTE_REPORT_FILE.replace("\\", "/")),
155             "errors": {},
156         }
157     try:
158         return {
159             "ok": True,
160             "source": "static-json",
161             "report": json.loads(ROBOT_ROUTE_REPORT_FILE.read_text(encoding="utf-8")),
162             "missing": None,
163             "errors": {},
164         }
165     except (OSError, json.JSONDecodeError) as exc:
166         return {
167             "ok": True,
168             "source": "fallback",
169             "report": None,
170             "missing": None,
171             "errors": {"robot_route_report": str(exc)},
172         }
173
174
175 def _load_orchestrator_status() -> dict:
```

```

176     if not ORCHESTRATOR_STATE_FILE.exists():
177         return {"ok": True, "progress": 0.0, "running_task": None, "completed": [], "failed": []}
178     try:
179         state = json.loads(ORCHESTRATOR_STATE_FILE.read_text(encoding="utf-8"))
180     except (OSError, json.JSONDecodeError) as exc:
181         return {"ok": False, "progress": 0.0, "running_task": None, "completed": [], "failed": [], "error": str(exc)}
182     task_status = state.get("task_status", {})
183     completed = [name for name, status in task_status.items() if status == "success"]
184     failed = [name for name, status in task_status.items() if status == "failed"]
185     running = [name for name, status in task_status.items() if status == "running"]
186     total = len(task_status)
187     return {
188         "ok": True,
189         "progress": round(len(completed) / total, 3) if total else 0.0,
190         "running_task": running[0] if running else None,
191         "completed": completed,
192         "failed": failed,
193         "task_status": task_status,
194     }
195
196
197 def _read_csv_rows(path: Path, missing_key: str, errors: dict, limitations: list[str]) -> list[dict]:
198     if not path.exists():
199         limitations.append(f"{missing_key}_missing")
200         return []
201     try:
202         with path.open("r", encoding="utf-8-sig", newline="") as handle:
203             return list(csv.DictReader(handle))
204     except OSError as exc:
205         errors[missing_key] = str(exc)
206         limitations.append(f"{missing_key}_unavailable")
207         return []
208
209
210 def _load_csv_payload(path: Path, missing_key: str, fields: list[str] | None = None, row_projector=None) -> dict:
211     errors = {}
212     limitations = []
213     rows = _read_csv_rows(path, missing_key, errors, limitations)
214     if row_projector is not None:
215         rows = [row_projector(row) for row in rows]
216     if fields is not None:
217         rows = [{field: row.get(field, "") for field in fields} for row in rows]
218     return {
219         "ok": True,
220         "source": "generated-csv" if rows else "fallback",
221         "items": rows,
222         "missing": [str(path).replace("\\", "/")] if limitations else [],
223         "errors": errors,
224     }
225

```

```
226
227 def _count_by_field(rows: list[dict], field: str) -> dict:
228     counts = {}
229     for row in rows:
230         value = (row.get(field) or "N/A").strip() or "N/A"
231         counts[value] = counts.get(value, 0) + 1
232     return counts
233
234
235 def _to_float(value) -> float | None:
236     try:
237         return float(value)
238     except (TypeError, ValueError):
239         return None
240
241
242 def _visualization_url(path: Path) -> str:
243     return "/static-outputs/visualizations/" + path.name
244
245
246 def _evidence_overlay_url(path: Path) -> str:
247     return "/static-outputs/evidence-overlays/" + path.name
248
249
250 def _candidate_kict_roots() -> list[Path]:
251     roots = []
252     env_root = os.environ.get("KICT_DATASET_ROOT")
253     if env_root:
254         roots.append(Path(env_root))
255     roots.extend(KICT_DATASET_ROOTS)
256     return roots
257
258
259 def _resolve_kict_artifact_path(value: str | None) -> Path | None:
260     if not value:
261         return None
262     raw_path = Path(value)
263     if raw_path.is_absolute() and raw_path.exists():
264         return raw_path
265     for root in _candidate_kict_roots():
266         candidate = root / raw_path
267         if candidate.exists():
268             return candidate
269     return None
270
271
272 def _make_overlay_image(image_path: Path, mask_path: Path, output_path: Path) -> None:
273     """Create a simple cyan mask overlay without changing the source dataset."""
274     output_path.parent.mkdir(parents=True, exist_ok=True)
275     image = Image.open(image_path).convert("RGB")
```

```

276     mask = Image.open(mask_path).convert("L")
277     if mask.size != image.size:
278         resampling = getattr(Image, "Resampling", Image).NEAREST
279         mask = mask.resize(image.size, resampling)
280     alpha = mask.point(lambda pixel: 110 if pixel > 0 else 0)
281     cyan_mask = Image.new("RGBA", image.size, (0, 188, 212, 0))
282     cyan_mask.putalpha(alpha)
283     overlay = Image.alpha_composite(image.convert("RGBA"), cyan_mask)
284     overlay.convert("RGB").save(output_path)
285
286
287 def _safe_artifact_stem(*parts: str | None) -> str:
288     joined = "_".join(part for part in parts if part)
289     return re.sub(r"^[A-Za-z0-9._-]+", "_", joined).strip(".") or "evidence"
290
291
292 def _select_kict_evidence_row(disease_id: str, last_inspection: str, kict_rows: list[dict]) -> dict | None:
293     matches = [row for row in kict_rows if row.get("disease_id") == disease_id]
294     if not matches:
295         return None
296     if last_inspection:
297         for row in reversed(matches):
298             if row.get("inspection_id") == last_inspection:
299                 return row
300     return matches[-1]
301
302
303 def _attach_recheck_evidence(priority_rechecks: list[dict], errors: dict, limitations: list[str]) -> list[dict]:
304     kict_rows = _read_csv_rows(ROBOT_KICT_FRAME_RECORDS_FILE, "robot_kict_frame_records", errors, limitations)
305     enriched = []
306     for row in priority_rechecks:
307         item = dict(row)
308         evidence = _select_kict_evidence_row(
309             item.get("disease_id", ""),
310             item.get("last_inspection", ""),
311             kict_rows,
312         )
313         if not evidence:
314             item["evidence_status"] = "missing_mapping"
315             enriched.append(item)
316             continue
317
318         image_path = _resolve_kict_artifact_path(evidence.get("kict_image_path") or evidence.get("kict_image_file"))
319         mask_path = _resolve_kict_artifact_path(evidence.get("kict_mask_path") or evidence.get("kict_mask_file"))
320         if not image_path or not mask_path:
321             item["evidence_status"] = "missing_source_file"
322             limitations.append("kict_source_file_missing")
323             enriched.append(item)
324             continue
325

```

```
326         output_name = _safe_artifact_stem(
327             evidence.get("disease_id"),
328             evidence.get("inspection_id"),
329             evidence.get("image_id"),
330             "overlay",
331         ) + ".png"
332         output_path = EVIDENCE_OVERLAY_ROOT / output_name
333         try:
334             if not output_path.exists():
335                 _make_overlay_image(image_path, mask_path, output_path)
336             item.update({
337                 "evidence_status": "ok",
338                 "evidence_overlay_url": _evidence_overlay_url(output_path),
339                 "evidence_image_id": evidence.get("image_id", ""),
340                 "evidence_inspection_id": evidence.get("inspection_id", ""),
341                 "evidence_mileage_text": evidence.get("mileage_text", ""),
342                 "evidence_clock_direction": evidence.get("clock_direction", ""),
343                 "evidence_image_file": evidence.get("kict_image_file") or evidence.get("kict_image_path", ""),
344                 "evidence_mask_file": evidence.get("kict_mask_file") or evidence.get("kict_mask_path", ""),
345             })
346         except OSError as exc:
347             item["evidence_status"] = "overlay_failed"
348             errors[f"evidence_overlay_{item.get('disease_id') or 'unknown'}"] = str(exc)
349         enriched.append(item)
350     return enriched
351
352
353 def _load_visualization_assets() -> dict:
354     missing = []
355     items = []
356     for key, path in VISUALIZATION_FILES.items():
357         if path.exists():
358             items.append({"key": key, "title": VISUALIZATION_TITLES[key], "url": _visualization_url(path)})
359         else:
360             missing.append(str(path).replace("\\", "/"))
361     return {
362         "ok": True,
363         "source": "generated-png" if items else "fallback",
364         "items": items,
365         "missing": missing,
366     }
367
368
369 def _load_algorithm_events() -> dict:
370     if not ALGORITHM_EVENTS_FILE.exists():
371         return {
372             "ok": True,
373             "source": "fallback",
374             "schema_version": "",
375             "generated_at": "",
```

```

376         "source_artifacts": [],
377         "events": [],
378         "missing": str(ALGORITHM_EVENTS_FILE.replace("\\", "/"),
379         "missing_message": ALGORITHM_EVENTS_MISSING_MESSAGE,
380         "errors": {},
381     }
382     try:
383         payload = json.loads(ALGORITHM_EVENTS_FILE.read_text(encoding="utf-8"))
384     except (OSError, json.JSONDecodeError) as exc:
385         return {
386             "ok": True,
387             "source": "error",
388             "schema_version": "",
389             "generated_at": "",
390             "source_artifacts": [],
391             "events": [],
392             "missing": "",
393             "missing_message": ALGORITHM_EVENTS_MISSING_MESSAGE,
394             "errors": {"algorithm_events": str(exc)},
395         }
396     schema_errors = _validate_algorithm_events_payload(payload)
397     if schema_errors:
398         return {
399             "ok": True,
400             "source": "error",
401             "schema_version": "",
402             "generated_at": "",
403             "source_artifacts": [],
404             "events": [],
405             "missing": "",
406             "missing_message": ALGORITHM_EVENTS_MISSING_MESSAGE,
407             "errors": {"algorithm_events": "; ".join(schema_errors)},
408         }
409     return {
410         "ok": True,
411         "source": "generated-json",
412         "schema_version": payload.get("schema_version", ""),
413         "generated_at": payload.get("generated_at", ""),
414         "source_artifacts": payload.get("source_artifacts", []),
415         "events": payload.get("events", []),
416         "missing": "",
417         "missing_message": "",
418         "errors": {},
419     }
420
421
422 def _validate_algorithm_artifact_ref(ref: object, label: str) -> list[str]:
423     if not isinstance(ref, dict):
424         return [f"{label} must be an object"]
425     errors = []

```

```
426     for field in ("label", "path", "exists"):
427         if field not in ref:
428             errors.append(f"{label} missing {field}")
429     if "exists" in ref and not isinstance(ref["exists"], bool):
430         errors.append(f"{label}.exists must be boolean")
431     return errors
432
433
434 def _validate_algorithm_events_payload(payload: object) -> list[str]:
435     if not isinstance(payload, dict):
436         return ["algorithm events payload must be an object"]
437     errors = []
438     if not payload.get("schema_version"):
439         errors.append("missing schema_version")
440     source_artifacts = payload.get("source_artifacts", [])
441     if not isinstance(source_artifacts, list):
442         errors.append("source_artifacts must be a list")
443     else:
444         for ref_index, ref in enumerate(source_artifacts, start=1):
445             errors.extend(_validate_algorithm_artifact_ref(ref, f"source_artifacts[{ref_index}]"))
446     events = payload.get("events")
447     if not isinstance(events, list):
448         errors.append("events must be a list")
449     return errors
450     required_event_fields = {"event_id", "stage", "title", "description", "inputs", "outputs", "claim_boundary", "status"}
451     for index, event in enumerate(events, start=1):
452         label = f"event {index}"
453         if not isinstance(event, dict):
454             errors.append(f"{label} must be an object")
455             continue
456         missing = sorted(required_event_fields - set(event))
457         if missing:
458             errors.append(f"{label} missing {' '.join(missing)}")
459         if event.get("status") not in ALGORITHM_EVENT_STATUS_VALUES:
460             errors.append(f"{label} has invalid status")
461         for side in ("inputs", "outputs"):
462             refs = event.get(side)
463             if not isinstance(refs, list):
464                 errors.append(f"{label}.{side} must be a list")
465             continue
466             for ref_index, ref in enumerate(refs, start=1):
467                 errors.extend(_validate_algorithm_artifact_ref(ref, f"{label}.{side}[{ref_index}]"))
468     return errors
469
470
471 def _safe_video_id(video_id: str | None) -> str:
472     safe = re.sub(r"[^A-Za-z0-9._-]+", "_", video_id or DEFAULT_VIDEO_ID).strip("._-")
473     return safe or DEFAULT_VIDEO_ID
474
475
```



```
476 def _video_url(video_id: str, filename: str) -> str:
477     return f"/static-video/{video_id}/{filename}"
478
479
480 def _video_table_payload(title: str, path: Path, missing: list[str], errors: dict, row_limit: int = 24) -> dict:
481     if not path.exists():
482         missing.append(str(path).replace("\\", "/"))
483     return {
484         "title": title,
485         "path": str(path).replace("\\", "/"),
486         "exists": False,
487         "columns": [],
488         "rows": [],
489         "row_count": 0,
490         "missing_message": VIDEO_MISSING_MESSAGE,
491     }
492     try:
493         with path.open("r", encoding="utf-8-sig", newline="") as handle:
494             rows = list(csv.DictReader(handle))
495     except (OSError, UnicodeDecodeError, csv.Error) as exc:
496         errors[title] = str(exc)
497     return {
498         "title": title,
499         "path": str(path).replace("\\", "/"),
500         "exists": False,
501         "columns": [],
502         "rows": [],
503         "row_count": 0,
504         "missing_message": VIDEO_MISSING_MESSAGE,
505     }
506     return {
507         "title": title,
508         "path": str(path).replace("\\", "/"),
509         "exists": True,
510         "columns": list(rows[0].keys()) if rows else [],
511         "rows": rows[:row_limit],
512         "row_count": len(rows),
513         "row_limit": row_limit,
514         "missing_message": "",
515     }
516
517
518 def _load_video_dashboard(video_id: str = DEFAULT_VIDEO_ID) -> dict:
519     safe_id = _safe_video_id(video_id)
520     missing: list[str] = []
521     errors: dict[str, str] = {}
522     video_files = [
523         {
524             "key": "demo_video",
525             "title": "原始 demo video",
```

```
526         "path": VIDEO_DATA_ROOT / f"{safe_id}.mp4",
527         "filename": f"{safe_id}.mp4",
528         "compatible_path": VIDEO_DATA_ROOT / f"{safe_id}.webm",
529         "compatible_filename": f"{safe_id}.webm",
530         "poster_path": VIDEO_FRAME_ROOT / safe_id / "frame_000001.jpg",
531         "poster_filename": "demo_poster.jpg",
532     },
533     {
534         "key": "opencv_annotated_video",
535         "title": "OpenCV annotated video",
536         "path": VIDEO_OUTPUT_ROOT / safe_id / "annotated_video.mp4",
537         "filename": "annotated_video.mp4",
538         "compatible_path": VIDEO_OUTPUT_ROOT / safe_id / "annotated_video.webm",
539         "compatible_filename": "annotated_video.webm",
540         "poster_path": VIDEO_OUTPUT_ROOT / safe_id / "annotated_frames" / "frame_000001.jpg",
541         "poster_filename": "annotated_poster.jpg",
542     },
543     {
544         "key": "supervision_annotated_video",
545         "title": "Supervision annotated video",
546         "path": VIDEO_OUTPUT_ROOT / safe_id / "supervision_annotated_video.mp4",
547         "filename": "supervision_annotated_video.mp4",
548         "compatible_path": VIDEO_OUTPUT_ROOT / safe_id / "supervision_annotated_video.webm",
549         "compatible_filename": "supervision_annotated_video.webm",
550         "poster_path": VIDEO_OUTPUT_ROOT / safe_id / "supervision_annotated_frames" / "frame_000001.jpg",
551         "poster_filename": "supervision_poster.jpg",
552     },
553 ]
554 videos = []
555 for item in video_files:
556     exists = item["path"].exists()
557     poster_exists = item["poster_path"].exists()
558     compatible_exists = item["compatible_path"].exists()
559     if not exists:
560         missing.append(str(item["path"]).replace("\\", "/"))
561     video_sources = []
562     if compatible_exists:
563         video_sources.append({
564             "url": _video_url(safe_id, item["compatible_filename"]),
565             "type": "video/webm",
566             "format": "webm",
567         })
568     if exists:
569         video_sources.append({
570             "url": _video_url(safe_id, item["filename"]),
571             "type": "video/mp4",
572             "format": "mp4",
573         })
574     videos.append({
575         "key": item["key"],
```

```

576         "title": item["title"],
577         "exists": exists,
578         "path": str(item["path"]).replace("\\", "/"),
579         "url": _video_url(safe_id, item["filename"]) if exists else "",
580         "compatible_exists": compatible_exists,
581         "compatible_path": str(item["compatible_path"]).replace("\\", "/"),
582         "compatible_url": _video_url(safe_id, item["compatible_filename"]) if compatible_exists else "",
583         "sources": video_sources,
584         "compatibility_note": (
585             "优先使用浏览器兼容 WebM 副本; 未生成时回退 MP4。"
586             if compatible_exists else
587             "未找到浏览器兼容 WebM 副本, 当前回退使用 MP4。"
588         ),
589         "poster_exists": poster_exists,
590         "poster_path": str(item["poster_path"]).replace("\\", "/"),
591         "poster_url": _video_url(safe_id, item["poster_filename"]) if poster_exists else "",
592         "missing_message": "" if exists else VIDEO_MISSING_MESSAGE,
593     })
594
595     tables = [
596         _video_table_payload("disease_features.csv", VIDEO_INSPECTION_ROOT / safe_id / "disease_features.csv", missing, errors),
597         _video_table_payload("inspection_sequence.csv", VIDEO_INSPECTION_ROOT / safe_id / "inspection_sequence.csv", missing, errors),
598         _video_table_payload("video_visualization_manifest.csv", VIDEO_OUTPUT_ROOT / safe_id / "video_visualization_manifest.csv", missing, errors),
599         _video_table_payload("supervision_visualization_manifest.csv", VIDEO_OUTPUT_ROOT / safe_id / "supervision_visualization_manifest.csv", missing, errors),
600     ]
601     return {
602         "ok": True,
603         "video_id": safe_id,
604         "source": "generated-video-artifacts" if any(video["exists"] for video in videos) or any(table["exists"] for table in tables) else "fallback",
605         "missing_message": VIDEO_MISSING_MESSAGE,
606         "boundary_notes": [
607             "tunnel_demo.mp4 是由 KICT 静态裂缝图像和 mask 合成的 demo video。",
608             "该视频用于验证视频输入 pipeline 和可视化展示。",
609             "它不是真实机器人连续巡检视频。",
610             "当前系统不是实时视频流分析系统。",
611             "supervision 只是可选可视化工具层, 不参与 Disease Memory Bank, no-id Association 和 Growth Analysis 核心逻辑。",
612         ],
613         "videos": videos,
614         "tables": tables,
615         "missing": missing,
616         "errors": errors,
617     }
618
619
620 def _project_credibility_row(row: dict, priority_output: bool = False) -> dict:
621     """Return a Web-safe copy without directional claims from unverified rows."""
622
623     item = dict(row)
624     verified = item.get("comparability_status") == "verified_comparable"
625     item["directional_metrics_available"] = verified

```

```

626     item["descriptive_area_relative_difference"] = ""
627     item["descriptive_area_difference_px"] = ""
628     if verified:
629         return item
630
631     item["descriptive_area_relative_difference"] = item.get("area_growth_rate", "")
632     item["descriptive_area_difference_px"] = item.get("area_growth_px", "")
633     item["growth_trend"] = (
634         "不可比较"
635         if priority_output or item.get("comparability_status") != "insufficient_history"
636         else "数据不足"
637     )
638     item["growth_description"] = (
639         f"静态面积审计: {item.get('first_area_px', '')} px□ -> "
640         f"{item.get('last_area_px', '')} px□。当前记录不构成方向性变化结论。"
641     )
642     if priority_output:
643         item["recheck_reason"] = "当前关注等级或风险等级满足复检条件; 纵向可比性尚未验证。"
644         item["recheck_suggestion"] = "建议记录静态面积证据, 并在取得真实复检数据后再判断方向性变化。"
645     item["area_growth_rate"] = ""
646     item["area_growth_px"] = ""
647     item["risk_level_change"] = ""
648     item["first_risk_level"] = ""
649     return item
650
651
652 def _inspection_count_from_growth(rows: list[dict]) -> int:
653     inspection_counts = [int(value) for value in (_to_float(row.get("inspection_count")) for row in rows) if value]
654     inspection_ids = {
655         row.get(field)
656         for row in rows
657         for field in ("first_inspection", "last_inspection")
658         if row.get(field)
659     }
660     return max(inspection_counts) if inspection_counts else len(inspection_ids)
661
662
663 def _load_project_summary() -> dict:
664     errors = {}
665     limitations = []
666     engineering_rows = _read_csv_rows(ENGINEERING_REPORT_FILE, "engineering_report", errors, limitations)
667     growth_rows = _read_csv_rows(DISEASE_GROWTH_FILE, "disease_growth_analysis", errors, limitations)
668     recheck_rows = _read_csv_rows(PRIORITY_RECHECK_FILE, "priority_recheck_list", errors, limitations)
669     visualizations = _load_visualization_assets()
670     disease_ids = {row.get("disease_id") for row in growth_rows if row.get("disease_id")}
671     high_risk_count = sum(1 for row in growth_rows if row.get("last_risk_level") == "高")
672     obvious_growth_count = sum(
673         1
674         for row in growth_rows
675         if row.get("comparability_status") == "verified_comparable"

```

```
676         and row.get("growth_trend") in {"明显增长", "持续增长"}
677     )
678     missing = [
679         str(path).replace("\\", "/")
680         for path in (ENGINEERING_REPORT_FILE, DISEASE_GROWTH_FILE, PRIORITY_RECHECK_FILE)
681         if not path.exists()
682     ] + visualizations["missing"]
683     return {
684         "ok": True,
685         "source": "generated-artifacts" if engineering_rows or growth_rows or recheck_rows else "fallback",
686         "inspection_count": _inspection_count_from_growth(growth_rows),
687         "disease_count": len(disease_ids),
688         "engineering_record_count": len(engineering_rows),
689         "priority_recheck_count": len(recheck_rows),
690         "high_risk_count": high_risk_count,
691         "obvious_growth_count": obvious_growth_count,
692         "visualization_count": len(visualizations["items"]),
693         "missing": missing,
694         "errors": errors,
695     }
696
697
698     AGENT_WORKBENCH_ROLES = {
699         "engineering_report": ("工程报告 Agent", "把逐帧观测整理为可复核的工程记录。"),
700         "growth_analysis": ("变化分析 Agent", "审查跨巡检可比性并生成变化证据。"),
701         "memory": ("病害记忆 Agent", "维护病害历史状态与可复用记忆。"),
702         "association": ("关联 Agent", "在不依赖 disease_id 的条件下关联历史观测。"),
703         "visualization": ("可视化 Agent", "生成图表、关系图和重点复检清单。"),
704         "final_report": ("报告 Agent", "汇总证据、限制和最终工程报告。"),
705     }
706
707     AGENT_WORKBENCH_LINKS = [
708         {"label": "工程报告", "view": "engineering-report"},
709         {"label": "增长分析", "view": "growth-analysis"},
710         {"label": "复检清单", "view": "recheck-list"},
711         {"label": "可视化图表", "view": "visualizations"},
712         {"label": "视频分析", "view": "video-analysis"},
713         {"label": "算法展示", "view": "algorithm-flow"},
714     ]
715
716     TERMINAL_RUN_STATUSES = {"success", "failed", "partial", "cancelled"}
717
718     def _workbench_output_metrics(outputs: object) -> dict[str, dict[str, object]]:
719         """Expose scalar output evidence without leaking local artifact paths."""
720
721         if not isinstance(outputs, dict):
722             return {}
723
724         def is_safe_scalar(key: object, value: object) -> bool:
725             if "path" in str(key).lower() or not isinstance(value, (str, int, float, bool)):
```

```
726         if not isinstance(value, str):
727             return True
728         normalized = value.strip()
729         return not (
730             re.match(r"^[A-Za-z]:[\\\/]", normalized)
731             or normalized.startswith(("/", "\\\\"))
732         )
733
734     projected: dict[str, dict[str, object]] = {}
735     for agent_name, raw_values in outputs.items():
736         if not isinstance(raw_values, dict):
737             continue
738         metrics = {
739             str(key): value
740             for key, value in raw_values.items()
741             if is_safe_scalar(key, value)
742         }
743         if metrics:
744             projected[str(agent_name)] = metrics
745     return projected
746
747
748 def _workbench_timeline(raw_timeline: object) -> list[dict[str, object]]:
749     """Collapse running/terminal events into one replayable entry per task."""
750
751     if not isinstance(raw_timeline, list):
752         return []
753     ordered_tasks: list[str] = []
754     events: dict[str, dict[str, object]] = {}
755     for raw_event in raw_timeline:
756         if not isinstance(raw_event, dict):
757             continue
758         task = str(raw_event.get("task") or "").strip()
759         if not task:
760             continue
761         if task not in events:
762             ordered_tasks.append(task)
763             events[task] = {
764                 "task": task,
765                 "status": "pending",
766                 "start_time": raw_event.get("start_time") or raw_event.get("time"),
767                 "end_time": None,
768                 "duration_seconds": None,
769                 "retry_count": 0,
770             }
771         event = events[task]
772         event["status"] = str(raw_event.get("status") or event["status"])
773         event["start_time"] = event.get("start_time") or raw_event.get("start_time") or raw_event.get("time")
774         event["end_time"] = raw_event.get("end_time") or event.get("end_time")
775         duration = raw_event.get("duration_seconds", raw_event.get("runtime_seconds"))
```

```
776         if isinstance(duration, (int, float)):
777             event["duration_seconds"] = round(float(duration), 4)
778             retry_count = raw_event.get("retry_count")
779             if isinstance(retry_count, int):
780                 event["retry_count"] = max(int(event["retry_count"]), retry_count)
781     return [events[task] for task in ordered_tasks]
782
783
784 def _workbench_run_status(metadata: dict, task_status: dict[str, str]) -> str:
785     values = {str(value).strip().lower() for value in task_status.values()}
786     metadata_status = str(metadata.get("status") or "").strip().lower()
787     if "failed" in values:
788         return "failed"
789     if "running" in values:
790         return "running"
791     if metadata_status in {"failed", "partial", "cancelled"}:
792         return metadata_status
793     if values and values == {"success"}:
794         return "success"
795     if values:
796         return "partial"
797     if metadata_status:
798         return metadata_status
799     if not task_status:
800         return "created"
801     return "partial"
802
803
804 def _load_agent_workbench(project_root: Path = ROOT, run_id: str | None = None) -> dict:
805     """Build a safe, read-only view of one orchestrator run and its evidence."""
806
807     manager = RunManager(Path(project_root))
808     run_summaries = manager.list_runs()
809     latest_run_id = manager.latest_run_id()
810     compact_runs = [
811         {
812             "run_id": item.get("run_id"),
813             "created_at": item.get("created_at"),
814             "updated_at": item.get("updated_at"),
815             "status": item.get("status"),
816             "completed_count": len(item.get("completed_tasks") or []),
817             "failed_count": len(item.get("failed_tasks") or []),
818         }
819         for item in reversed(run_summaries)
820     ]
821     if run_id and not re.fullmatch(r"run_\d{3,}", run_id):
822         return {
823             "ok": False,
824             "error_code": "run_not_found",
825             "error": f"run not found: {run_id}",
```

```
826     }
827     if not latest_run_id:
828         if run_id:
829             return {
830                 "ok": False,
831                 "error_code": "run_not_found",
832                 "error": f"run not found: {run_id}",
833             }
834         return {
835             "ok": True,
836             "source": "fallback",
837             "selected_run": None,
838             "runs": [],
839             "dag": {"nodes": [], "edges": []},
840             "timeline": [],
841             "outputs": {},
842             "current_results": {
843                 "binding_status": "unavailable",
844                 "bound_run_id": None,
845                 "summary": {},
846                 "links": [],
847                 "limitations": ["no_run_history"],
848             },
849             "missing": ["runs"],
850             "errors": {},
851             "poll_interval_ms": None,
852         }
853
854     selected_run_id = run_id or latest_run_id
855     if not re.fullmatch(r"run_\d{3,}", selected_run_id) or selected_run_id not in [
856         item.get("run_id") for item in run_summaries
857     ]:
858         return {
859             "ok": False,
860             "error_code": "run_not_found",
861             "error": f"run not found: {selected_run_id}",
862         }
863
864     selected = manager.load_run(selected_run_id)
865     metadata = selected.get("metadata") if isinstance(selected.get("metadata"), dict) else {}
866     state = selected.get("state") if isinstance(selected.get("state"), dict) else {}
867     dag = selected.get("dag") if isinstance(selected.get("dag"), dict) else {}
868     raw_task_status = state.get("task_status", {})
869     task_status = raw_task_status if isinstance(raw_task_status, dict) else {}
870     task_status = {
871         str(key): str(value).strip().lower()
872         for key, value in task_status.items()
873     }
874     timeline = _workbench_timeline(selected.get("timeline"))
875     timeline_by_task = {str(item["task"]): item for item in timeline}
```



```

876     context = state.get("context_snapshot", {})
877     context = context if isinstance(context, dict) else {}
878     outputs = _workbench_output_metrics(context.get("outputs"))
879
880     raw_nodes = dag.get("nodes", [])
881     raw_nodes = raw_nodes if isinstance(raw_nodes, list) else []
882     nodes = []
883     for raw_node in raw_nodes:
884         if not isinstance(raw_node, dict):
885             continue
886         node_id = str(raw_node.get("id") or raw_node.get("agent") or "").strip()
887         if not node_id:
888             continue
889         agent_name = str(raw_node.get("agent") or node_id)
890         role_title, role_description = AGENT_WORKBENCH_ROLES.get(
891             agent_name,
892             (agent_name.replace("_", " ").title(), "执行编排任务并输出结构化证据。"),
893         )
894         timeline_item = timeline_by_task.get(node_id, {})
895         nodes.append(
896             {
897                 "id": node_id,
898                 "agent": agent_name,
899                 "title": role_title,
900                 "description": role_description,
901                 "status": task_status.get(node_id, str(raw_node.get("status") or "pending")),
902                 "duration_seconds": timeline_item.get("duration_seconds"),
903                 "retry_count": timeline_item.get("retry_count", 0),
904                 "metrics": outputs.get(agent_name, outputs.get(node_id, {})),
905             }
906         )
907
908     node_ids = {node["id"] for node in nodes}
909     raw_edges = dag.get("edges", [])
910     raw_edges = raw_edges if isinstance(raw_edges, list) else []
911     edges = [
912         {"source": str(edge.get("source")), "target": str(edge.get("target"))}
913         for edge in raw_edges
914         if isinstance(edge, dict)
915         and str(edge.get("source")) in node_ids
916         and str(edge.get("target")) in node_ids
917     ]
918
919     missing = []
920     errors = {}
921     selected_dir = manager.runs_dir / selected_run_id
922     for filename, payload in (
923         ("metadata.json", metadata),
924         ("state.json", state),
925         ("dag.json", dag),

```

```

926 ):
927     if not (selected_dir / filename).exists():
928         missing.append(filename)
929     elif not payload:
930         errors[filename] = "unavailable_or_invalid"
931
932 total_tasks = len(task_status) or len(nodes)
933 completed_count = sum(1 for value in task_status.values() if value == "success")
934 failed_count = sum(1 for value in task_status.values() if value == "failed")
935 running_count = sum(1 for value in task_status.values() if value == "running")
936 run_status = _workbench_run_status(metadata, task_status)
937 runtime_values = [
938     item.get("duration_seconds")
939     for item in timeline
940     if isinstance(item.get("duration_seconds"), (int, float))
941 ]
942 is_latest = selected_run_id == latest_run_id
943
944 if is_latest:
945     project_summary = _load_project_summary()
946     current_results = {
947         "binding_status": "matched_latest",
948         "bound_run_id": latest_run_id,
949         "summary": {
950             key: value
951             for key, value in project_summary.items()
952             if key
953             in [
954                 "source",
955                 "inspection_count",
956                 "disease_count",
957                 "engineering_record_count",
958                 "priority_recheck_count",
959                 "high_risk_count",
960                 "obvious_growth_count",
961                 "visualization_count",
962             ]
963         },
964         "links": AGENT_WORKBENCH_LINKS,
965         "limitations": [
966             "KICT 图像与 mask 为真实数据；巡检时间、里程和跨巡检关系包含仿真元数据。",
967             "结果属于规则证据与图像证据，仍需人工或传感器复核。",
968         ],
969         "missing": [
970             Path(str(value).replace("\\", "/")).name
971             for value in project_summary.get("missing", [])
972         ],
973         "errors": sorted(project_summary.get("errors", [])),
974     }
975 else:

```

```
976         current_results = {
977             "binding_status": "execution_only",
978             "bound_run_id": None,
979             "summary": {},
980             "links": [],
981             "limitations": [
982                 "历史 run 只回放执行状态、DAG、时间线和快照指标。",
983                 "当前图片与 CSV 未按历史 run 冻结，因此不会绑定到本次回放。",
984             ],
985             "missing": [],
986             "errors": [],
987         }
988
989     return {
990         "ok": True,
991         "source": "run-artifacts",
992         "selected_run": {
993             "run_id": selected_run_id,
994             "is_latest": is_latest,
995             "status": run_status,
996             "created_at": metadata.get("created_at"),
997             "updated_at": metadata.get("updated_at"),
998             "progress": round(completed_count / total_tasks, 3) if total_tasks else 0.0,
999             "agent_count": total_tasks,
1000             "completed_count": completed_count,
1001             "failed_count": failed_count,
1002             "running_count": running_count,
1003             "total_runtime_seconds": round(sum(runtime_values), 4),
1004         },
1005         "runs": compact_runs,
1006         "dag": {"nodes": nodes, "edges": edges},
1007         "timeline": timeline,
1008         "outputs": outputs,
1009         "current_results": current_results,
1010         "missing": missing,
1011         "errors": errors,
1012         "poll_interval_ms": 2000 if is_latest and run_status not in TERMINAL_RUN_STATUSES else None,
1013     }
1014
1015
1016 ENGINEERING_REPORT_FIELDS = [
1017     "inspection_id",
1018     "disease_id",
1019     "disease_type",
1020     "start_mileage_text",
1021     "end_mileage_text",
1022     "start_ring",
1023     "end_ring",
1024     "main_clock_direction",
1025     "frame_count",
```

```
1026     "max_area_px",
1027     "mean_area_px",
1028     "risk_level",
1029     "engineering_description",
1030     "representative_image_path",
1031     "representative_mask_path",
1032 ]
1033 GROWTH_ANALYSIS_FIELDS = [
1034     "disease_id",
1035     "disease_type",
1036     "first_inspection",
1037     "last_inspection",
1038     "first_area_px",
1039     "last_area_px",
1040     "area_growth_rate",
1041     "area_growth_px",
1042     "first_risk_level",
1043     "last_risk_level",
1044     "growth_trend",
1045     "attention_level",
1046     "last_mileage_range",
1047     "main_clock_direction",
1048     "growth_description",
1049     "comparability_status",
1050     "directional_metrics_available",
1051     "descriptive_area_relative_difference",
1052     "descriptive_area_difference_px",
1053 ]
1054 RECHECK_FIELDS = [
1055     "priority_rank",
1056     "disease_id",
1057     "disease_type",
1058     "attention_level",
1059     "growth_trend",
1060     "area_growth_rate",
1061     "first_risk_level",
1062     "last_risk_level",
1063     "last_mileage_range",
1064     "main_clock_direction",
1065     "recheck_reason",
1066     "recheck_suggestion",
1067     "growth_description",
1068     "comparability_status",
1069     "directional_metrics_available",
1070     "descriptive_area_relative_difference",
1071     "descriptive_area_difference_px",
1072 ]
1073
1074
1075 def _load_robot_dashboard() -> dict:
```

```
1076     errors = []
1077     limitations = ["rule_evidence", "requires_review", "simulation_or_coarse_location"]
1078     route_payload = _load_robot_route_report()
1079     if route_payload.get("missing"):
1080         limitations.append("robot_route_report_missing")
1081     errors.update(route_payload.get("errors", {}))
1082
1083     # CSV tables are generated by the robot analysis scripts and are optional for demo startup.
1084     priority_rechecks = [
1085         _project_credibility_row(row, priority_output=True)
1086         for row in _read_csv_rows(PRIORITY_RECHECK_FILE, "priority_recheck_list", errors, limitations)
1087     ]
1088     growth_rows = [
1089         _project_credibility_row(row)
1090         for row in _read_csv_rows(DISEASE_GROWTH_FILE, "disease_growth_analysis", errors, limitations)
1091     ]
1092     priority_rechecks.sort(key=lambda row: int(_to_float(row.get("priority_rank")) or 999999))
1093     priority_rechecks = _attach_recheck_evidence(priority_rechecks, errors, limitations)
1094
1095     route_report = route_payload.get("report") or {}
1096     route_summary = route_report.get("summary") or {}
1097     high_risk_count = sum(1 for row in growth_rows if row.get("last_risk_level") == "高")
1098     summary = {
1099         "inspection_count": _inspection_count_from_growth(growth_rows),
1100         "frame_count": route_summary.get("frame_count"),
1101         "track_count": route_summary.get("track_count") or len(growth_rows),
1102         "review_count": route_summary.get("review_count") or len(priority_rechecks),
1103         "priority_recheck_count": len(priority_rechecks),
1104         "high_risk_count": high_risk_count,
1105         "growth_object_count": len(growth_rows),
1106     }
1107
1108     visualization_links = {}
1109     for key, path in VISUALIZATION_FILES.items():
1110         if path.exists():
1111             visualization_links[key] = _visualization_url(path)
1112         else:
1113             limitations.append(f"{key}_visualization_missing")
1114
1115     return {
1116         "ok": True,
1117         "source": "generated-artifacts" if priority_rechecks or growth_rows or route_report else "fallback",
1118         "summary": summary,
1119         "route_report": route_report or None,
1120         "priority_rechecks": priority_rechecks,
1121         "growth_distribution": _count_by_field(growth_rows, "growth_trend"),
1122         "attention_distribution": _count_by_field(growth_rows, "attention_level"),
1123         "disease_type_distribution": _count_by_field(growth_rows, "disease_type"),
1124         "risk_level_distribution": _count_by_field(growth_rows, "last_risk_level"),
1125         "visualization_links": visualization_links,
```

```

1126         "limitations": sorted(set(limitations + route_summary.get("limitations", []))),
1127         "errors": errors,
1128     }
1129
1130
1131 def _load_model_once():
1132     global _MODEL, _CONFIG
1133     if _MODEL is None or _CONFIG is None:
1134         _MODEL, _CONFIG = load_model(
1135             model_source=_WEB_MODEL_CONFIG.model_source,
1136             segformer_config=_WEB_MODEL_CONFIG.segformer_config,
1137             segformer_checkpoint=_WEB_MODEL_CONFIG.segformer_checkpoint,
1138             segformer_repo_root=_WEB_MODEL_CONFIG.segformer_repo_root,
1139             segformer_device=_WEB_MODEL_CONFIG.segformer_device,
1140         )
1141     return _MODEL, _CONFIG
1142
1143
1144 def _parse_upload(body: bytes, content_type: str) -> tuple[str, bytes]:
1145     header = f"Content-Type: {content_type}\r\nMIME-Version: 1.0\r\n\r\n".encode("utf-8")
1146     message = BytesParser(policy=policy.default).parsebytes(header + body)
1147     if not message.is_multipart():
1148         raise ValueError("Expected multipart/form-data upload.")
1149
1150     for part in message.iter_parts():
1151         if part.get_param("name", header="content-disposition") != "image":
1152             continue
1153         filename = part.get_filename() or "upload.jpg"
1154         payload = part.get_payload(decode=True) or b""
1155         if not payload:
1156             raise ValueError("Uploaded image is empty.")
1157         return filename, payload
1158
1159     raise ValueError("Missing form field: image.")
1160
1161
1162 def _artifact_url(job_id: str, filename: str) -> str:
1163     return f"/outputs/{job_id}/{filename}"
1164
1165
1166 def _view_payload(report: dict, job_id: str, original_name: str) -> list[dict[str, str]]:
1167     artifacts = report.get("artifacts", {})
1168     uncertainty = report.get("uncertainty_summary", {})
1169     disagreement = report.get("disagreement_summary", {})
1170     return [
1171         {"label": "Original", "url": _artifact_url(job_id, original_name)},
1172         {"label": "Single mask", "url": _artifact_url(job_id, artifacts["single_mask"])},
1173         {"label": "Fused mask", "url": _artifact_url(job_id, artifacts["fused_mask"])},
1174         {"label": "Selected mask", "url": _artifact_url(job_id, artifacts["selected_mask"])},
1175         {"label": "Overlay", "url": _artifact_url(job_id, artifacts["overlay"])},

```

```
1176         {"label": "Selected overlay", "url": _artifact_url(job_id, artifacts["selected_overlay"])},
1177         {
1178             "label": "Uncertainty",
1179             "url": _artifact_url(job_id, artifacts["uncertainty_heatmap"]),
1180             "available": bool(uncertainty.get("available", True)),
1181             "reason": uncertainty.get("reason", ""),
1182         },
1183         {
1184             "label": "Disagreement",
1185             "url": _artifact_url(job_id, artifacts["disagreement_heatmap"]),
1186             "available": bool(disagreement.get("available", True)),
1187             "reason": disagreement.get("reason", ""),
1188         },
1189         {"label": "Skeleton", "url": _artifact_url(job_id, artifacts["skeleton"])},
1190     ]
1191
1192
1193 def _detect_image(filename: str, data: bytes, tta_mode: str) -> dict:
1194     safe_name = _safe_filename(filename)
1195     suffix = Path(safe_name).suffix.lower()
1196     if suffix not in IMAGE_SUFFIXES:
1197         raise ValueError(f"Unsupported image suffix: {suffix}")
1198
1199     job_id = time.strftime("%Y%m%d-%H%M%S-") + uuid.uuid4().hex[:8]
1200     job_dir = OUTPUT_ROOT / job_id
1201     job_dir.mkdir(parents=True, exist_ok=True)
1202
1203     original_path = job_dir / safe_name
1204     original_path.write_bytes(data)
1205
1206     started = time.perf_counter()
1207     with _INFER_LOCK:
1208         model, config = _load_model_once()
1209         report = process_image(model, config, original_path, job_dir, tta_mode=tta_mode)
1210     elapsed_ms = int((time.perf_counter() - started) * 1000)
1211
1212     report["job_id"] = job_id
1213     report["elapsed_ms"] = elapsed_ms
1214     report["original_artifact"] = safe_name
1215     report["views"] = _view_payload(report, job_id, safe_name)
1216     report["report_url"] = _artifact_url(job_id, report["artifacts"]["report"])
1217     inspection_report = build_inspection_report(
1218         report,
1219         job_id=job_id,
1220         original_name=safe_name,
1221         output_dir=job_dir,
1222     )
1223     report["inspection_report"] = inspection_report
1224     report["inspection_report_url"] = inspection_report["inspection_report_url"]
1225     return report
```

```
1226
1227
1228 class DetectionHandler(SimpleHTTPRequestHandler):
1229     def log_message(self, format: str, *args) -> None:
1230         print(f"[web_app] {self.address_string()} - {format % args}")
1231
1232     def _send(self, status: int, data: bytes, content_type: str) -> None:
1233         self.send_response(status)
1234         self.send_header("Content-Type", content_type)
1235         self.send_header("Content-Length", str(len(data)))
1236         self.end_headers()
1237         self.wfile.write(data)
1238
1239     def _send_json(self, payload: dict, status: HTTPStatus = HTTPStatus.OK) -> None:
1240         code, data, content_type = _json_bytes(payload, status)
1241         self._send(code, data, content_type)
1242
1243     def do_GET(self) -> None:
1244         parsed = urlparse(self.path)
1245         path = unquote(parsed.path)
1246
1247         if path in {"/", "/index.html"}:
1248             self._serve_file(WEB_ROOT / "index.html")
1249             return
1250         if path in {"/platform", "/platform/"}:
1251             self._serve_file(WEB_PLATFORM_ROOT / "dashboard" / "index.html")
1252             return
1253         if path.startswith("/web/"):
1254             self._serve_file(WEB_PLATFORM_ROOT / _strip_prefix(path, "/web/"))
1255             return
1256         if path.startswith("/assets/"):
1257             self._serve_file(WEB_ROOT / path.lstrip("/"))
1258             return
1259         if path.startswith("/static-outputs/visualizations/"):
1260             self._serve_visualization_file(_strip_prefix(path, "/static-outputs/visualizations/"))
1261             return
1262         if path.startswith("/static-outputs/evidence-overlays/"):
1263             self._serve_evidence_overlay_file(_strip_prefix(path, "/static-outputs/evidence-overlays/"))
1264             return
1265         if path.startswith("/static-video/"):
1266             video_path = _strip_prefix(path, "/static-video/")
1267             parts = [part for part in video_path.split("/") if part]
1268             if len(parts) != 2:
1269                 self._send_json({"error": "Not found"}, HTTPStatus.NOT_FOUND)
1270                 return
1271             self._serve_video_file(parts[0], parts[1])
1272             return
1273         if path.startswith("/outputs/"):
1274             self._serve_file(OUTPUT_ROOT / _strip_prefix(path, "/outputs/"))
1275             return
```



```
1276         if path == "/api/health":
1277             self._send_json(
1278                 {
1279                     "ok": True,
1280                     "model_loaded": _MODEL is not None,
1281                     "model_source": _WEB_MODEL_CONFIG.model_source,
1282                     "segformer_device": _WEB_MODEL_CONFIG.segformer_device,
1283                 }
1284             )
1285             return
1286         if path == "/api/status":
1287             self._send_json(_load_orchestrator_status())
1288             return
1289         if path == "/api/dag":
1290             self._send_json(dag_payload(ROOT))
1291             return
1292         if path == "/api/runs":
1293             self._send_json(runs_payload(ROOT))
1294             return
1295         if path == "/api/agent-workbench":
1296             query = parse_qs(parsed.query)
1297             requested_run = (query.get("run_id") or [None])[0]
1298             payload = _load_agent_workbench(ROOT, requested_run)
1299             status = HTTPStatus.NOT_FOUND if payload.get("error_code") == "run_not_found" else HTTPStatus.OK
1300             self._send_json(payload, status)
1301             return
1302         if path.startswith("/api/run/"):
1303             self._send_json(run_payload(ROOT, Path(strip_prefix(path, "/api/run/")).name))
1304             return
1305         if path == "/api/evidence":
1306             self._send_json(_load_evidence_payload())
1307             return
1308         if path == "/api/robot-route-report":
1309             self._send_json(_load_robot_route_report())
1310             return
1311         if path == "/api/robot-dashboard":
1312             self._send_json(_load_robot_dashboard())
1313             return
1314         if path == "/api/project-summary":
1315             self._send_json(_load_project_summary())
1316             return
1317         if path == "/api/engineering-report":
1318             self._send_json(_load_csv_payload(ENGINEERING_REPORT_FILE, "engineering_report", ENGINEERING_REPORT_FIELDS))
1319             return
1320         if path == "/api/growth-analysis":
1321             self._send_json(
1322                 _load_csv_payload(
1323                     DISEASE_GROWTH_FILE,
1324                     "disease_growth_analysis",
1325                     GROWTH_ANALYSIS_FIELDS,
```

```
1326         _project_credibility_row,
1327     )
1328 )
1329 return
1330 if path == "/api/recheck-list":
1331     self._send_json(
1332         _load_csv_payload(
1333             PRIORITY_RECHECK_FILE,
1334             "priority_recheck_list",
1335             RECHECK_FIELDS,
1336             lambda row: _project_credibility_row(row, priority_output=True),
1337         )
1338     )
1339 return
1340 if path == "/api/visualization-assets":
1341     self._send_json(_load_visualization_assets())
1342     return
1343 if path == "/api/algorithm-events":
1344     self._send_json(_load_algorithm_events())
1345     return
1346 if path == "/api/video-dashboard":
1347     self._send_json(_load_video_dashboard(DEFAULT_VIDEO_ID))
1348     return
1349
1350 self._send_json({"error": "Not found"}, HTTPStatus.NOT_FOUND)
1351
1352 def do_POST(self) -> None:
1353     parsed = urlparse(self.path)
1354     if parsed.path != "/api/detect":
1355         self._send_json({"error": "Not found"}, HTTPStatus.NOT_FOUND)
1356         return
1357
1358     try:
1359         length = int(self.headers.get("Content-Length", "0"))
1360         if length <= 0:
1361             raise ValueError("Missing request body.")
1362         if length > MAX_UPLOAD_BYTES:
1363             raise ValueError(f"Upload exceeds {MAX_UPLOAD_BYTES // (1024 * 1024)} MB limit.")
1364
1365         content_type = self.headers.get("Content-Type", "")
1366         body = self.rfile.read(length)
1367         filename, data = _parse_upload(body, content_type)
1368         report = _detect_image(filename, data, tta_mode="light")
1369         self._send_json({"ok": True, "report": report})
1370     except Exception as exc:
1371         self._send_json({"ok": False, "error": str(exc)}, HTTPStatus.BAD_REQUEST)
1372
1373 def _serve_file(self, path: Path) -> None:
1374     resolved = path.resolve()
1375     allowed_roots = [
```

```

1376         WEB_ROOT.resolve(),
1377         WEB_PLATFORM_ROOT.resolve(),
1378         OUTPUT_ROOT.resolve(),
1379         VISUALIZATION_ROOT.resolve(),
1380         EVIDENCE_OVERLAY_ROOT.resolve(),
1381         VIDEO_DATA_ROOT.resolve(),
1382         VIDEO_FRAME_ROOT.resolve(),
1383         VIDEO_OUTPUT_ROOT.resolve(),
1384     ]
1385     if not any(_is_relative_to(resolved, root) for root in allowed_roots):
1386         self._send_json({"error": "Forbidden"}, HTTPStatus.FORBIDDEN)
1387         return
1388     if not resolved.exists() or not resolved.is_file():
1389         self._send_json({"error": "Not found"}, HTTPStatus.NOT_FOUND)
1390         return
1391
1392     content_type = mimetypes.guess_type(str(resolved))[0] or "application/octet-stream"
1393     self.send_response(HTTPStatus.OK)
1394     self.send_header("Content-Type", content_type)
1395     self.send_header("Content-Length", str(resolved.stat().st_size))
1396     self.end_headers()
1397     with resolved.open("rb") as fh:
1398         shutil.copyfileobj(fh, self.wfile)
1399
1400     def _serve_visualization_file(self, filename: str) -> None:
1401         safe_name = Path(filename).name
1402         path = VISUALIZATION_ROOT / safe_name
1403         if filename != safe_name or path.suffix.lower() != ".png":
1404             self._send_json({"error": "Not found"}, HTTPStatus.NOT_FOUND)
1405             return
1406         self._serve_file(path)
1407
1408     def _serve_evidence_overlay_file(self, filename: str) -> None:
1409         safe_name = Path(filename).name
1410         path = EVIDENCE_OVERLAY_ROOT / safe_name
1411         if filename != safe_name or path.suffix.lower() != ".png":
1412             self._send_json({"error": "Not found"}, HTTPStatus.NOT_FOUND)
1413             return
1414         self._serve_file(path)
1415
1416     def _serve_video_file(self, video_id: str, filename: str) -> None:
1417         safe_id = _safe_video_id(video_id)
1418         safe_name = Path(filename).name
1419         suffix = Path(safe_name).suffix.lower()
1420         if video_id != safe_id or filename != safe_name or suffix not in {".mp4", ".webm", ".jpg"}:
1421             self._send_json({"error": "Not found"}, HTTPStatus.NOT_FOUND)
1422             return
1423         if safe_name == f"{safe_id}.mp4":
1424             path = VIDEO_DATA_ROOT / safe_name
1425         elif safe_name == f"{safe_id}.webm":

```

```

1426         path = VIDEO_DATA_ROOT / safe_name
1427     elif safe_name in {"annotated_video.mp4", "supervision_annotated_video.mp4"}:
1428         path = VIDEO_OUTPUT_ROOT / safe_id / safe_name
1429     elif safe_name in {"annotated_video.webm", "supervision_annotated_video.webm"}:
1430         path = VIDEO_OUTPUT_ROOT / safe_id / safe_name
1431     elif safe_name == "demo_poster.jpg":
1432         path = VIDEO_FRAME_ROOT / safe_id / "frame_000001.jpg"
1433     elif safe_name == "annotated_poster.jpg":
1434         path = VIDEO_OUTPUT_ROOT / safe_id / "annotated_frames" / "frame_000001.jpg"
1435     elif safe_name == "supervision_poster.jpg":
1436         path = VIDEO_OUTPUT_ROOT / safe_id / "supervision_annotated_frames" / "frame_000001.jpg"
1437     else:
1438         self._send_json({"error": "Not found"}, HTTPStatus.NOT_FOUND)
1439         return
1440     self._serve_file(path)
1441
1442
1443 def parse_args() -> argparse.Namespace:
1444     parser = argparse.ArgumentParser(description="Run the tunnel defect live detection web app.")
1445     parser.add_argument("--host", default="127.0.0.1")
1446     parser.add_argument("--port", type=int, default=8000)
1447     parser.add_argument("--model-source", choices=["legacy", "segformer"], default=WebModelConfig.model_source)
1448     parser.add_argument("--python-executable", type=Path, default=WebModelConfig.python_executable)
1449     parser.add_argument("--segformer-config", type=Path, default=DEFAULT_SEGFORMER_CONFIG)
1450     parser.add_argument("--segformer-checkpoint", type=Path, default=DEFAULT_SEGFORMER_CHECKPOINT)
1451     parser.add_argument("--segformer-repo-root", type=Path, default=DEFAULT_SEGFORMER_REPO_ROOT)
1452     parser.add_argument("--segformer-device", default=WebModelConfig.segformer_device)
1453     return parser.parse_args()
1454
1455
1456 def main() -> None:
1457     global _WEB_MODEL_CONFIG
1458     args = parse_args()
1459     _WEB_MODEL_CONFIG = WebModelConfig(
1460         model_source=args.model_source,
1461         python_executable=args.python_executable,
1462         segformer_config=args.segformer_config,
1463         segformer_checkpoint=args.segformer_checkpoint,
1464         segformer_repo_root=args.segformer_repo_root,
1465         segformer_device=args.segformer_device,
1466     )
1467     OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)
1468     server = ThreadingHTTPServer((args.host, args.port), DetectionHandler)
1469     print(f"Serving live detector at http://{args.host}:{args.port}")
1470     print(f"Model source: {_WEB_MODEL_CONFIG.model_source}")
1471     server.serve_forever()
1472
1473
1474 if __name__ == "__main__":
1475     main()

```